# Machine Learning project report

Michele Nicoletti - 1886646 - nicoletti.1886646@studenti.uniroma1.it
Lorenzo Pecorari - 1885161 - pecorari.1885161@studenti.uniroma1.it

June 14, 2025

## 1 Introduction

The following report aims to describe different attempts of implementing a reinforcement learning agent for solving a run of "Taxi-v3", an environment offered by the Gymnasium Documentation. The first of them uses a Tabular Q-Learning approach while the second ones can rely on the concept of neural network. They differ in general aspects, metrics and computational effort for the machine that runs each of them but with the same scope.

Each solution is developed by using Python with Pytorch, a library that allows to create tools usable by the agents. In combination with that, are used other useful libraries as Numpy and Matplotlib, for data manipulation and result plotting.

For each solution, it will be provided the implementation and the results about the experiment; in particular, there will be showed, where possible, the curves of accuracy, loss and rewards obtained by the agents.

## 2 Taxi-v3, a Toy Text environment

Graphically, it consists in a 2D fancy representation of a taxi that has to pickup a passenger and drop it in a destination. More in depth, it represents a 500 discrete states grid world generated by 25 positions for the taxi, 5 for the passenger and 4 for the building/destination. In particular, each destination is represented by a color among red, green, yellow and blue and the passenger can be in one of them or inside the taxi.

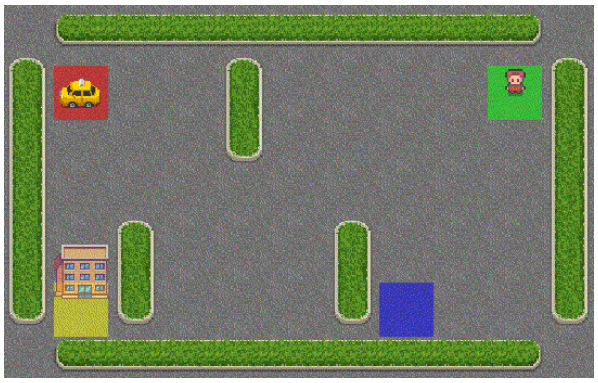


Figure 1: Graphical representation of the environment

Each time, the taxi can execute one of the 6 available moves: up, down, left, right, pick up, drop off. For a correct delivery of the passenger there will be a positive reward of 20, while an illegal pick up or drop off determines a loss of 10; for any step different from the previous actions the award will be -1.

There can be at most 300 possible initial states in which the taxi, the destination and the passenger are in different locations.

For avoiding an undefined number of actions executed during a single run, it is set by default a maximum of 200 steps per episode determining a termination state equal to "terminated" or "truncated" if the delivery happens or not.

By that, the scope of the project is to create and train a model allowing the agent to correctly solve the environment.

The environment is imported by the library `gymnasium` through the method `gymnasium.make("Taxi-v3")`. For more details, the official page of the environment is available here: `https://gymnasium.farama.org/environments/toy_text/taxi/`

# 3 First solution: Tabular Q-Learning

## 3.1 Idea behind the solution

The following implementation uses a table where values are stored, accessed and updated each time an action is taken and executed. More in detail, using the concepts of Q-values and "epsilon-greedy" approach, the agent is able to choice the better action in each state it will be by accessing the correct element of the table.

The structure of the table is characterized by a $500 \times 6$ matrix, where each row is related to a possible state of the agent and each column is the relative action among the possible ones. Each element of the structure contains the Q-value of the given state and action: it is the quality of their combination.

| Q | a0 | a1 | ... | an |
|---|---|---|---|---|
| s0 | ... | ... | ... | ... |
| s1 | ... | ... | ... | ... |
| ... | ... | ... | ... | ... |
| sm | ... | ... | ... | ... |

Figure 2: Graphical representation of the table

Initially, each value of the matrix is set to 0 but, with the update method, it will be gradually populated by different values obtained by the Q-function:

$$Q(s,a) = (1-\alpha) \cdot Q(s,a) + \alpha(r + \gamma \cdot \max_{a' \in A}(Q(s',a')))$$

where $r$ is the instant reward, obtained executing $a$ in $s$, $\alpha \in (0,1]$ is the learning rate, $\gamma$ is the discount factor in the range [0, 1), $s'$ is the next state and $a'$ is the action to be performed in the next state (maximizing the Q-value).

Even if the environment is deterministic, it has been also tested as it was nondeterministic just by changing the value of the learning rate $\alpha$. For the deterministic case, the value of $\alpha$ has been set to 1, leading to the following formula:

$$Q(s,a) = r + \gamma \cdot \max_{a' \in A}(Q(s',a'))$$

On the other hand, the nondeterministic situation is characterized by $\alpha \in (0,1)$ that allows to define the importance of the previous value of $Q(s,a)$ with respect to the new one. This is applied since in nondeterministic environments there are stochastic transitions and rewards.

Each time that in a state $s$ is performed an action $a$, their relative Q-value is updated with the formulas reported above, taking in consideration also the future values obtainable in the next state.

The relevance of the possible future rewards with respect to the current is defined by the discount factor $\gamma$: the higher the value the more the agent will prefer actions maximizing future rewards and viceversa.

## 3.2 Implementation

This solution needs of two different classes: one for handling the table and one for the agent.

The class `QTable` is the matrix representing the table and it is initialized through `np.zeros((states, actions))`, where `states` and `actions` are referred to the rows and columns of it. The update is

performed by `QTable.update()` that uses the formula previously showed.

The class `Agent` generates the objects in charge to solve the environment. It uses the table through `QTable` and also stores the parameters $\epsilon$, $\epsilon_{dec}$, $\epsilon_{min}$, $\gamma$ and $\alpha$. The main functions are:

- `choose_action()`, which picks a random float and if it is lower than $\epsilon$ it samples an action, otherwise it chooses the action returning the highest reward given the current state;
- `update_table()`, that gets the current Q-value and updates it with the one coming by the application of the previously defined formula;
- `train()`, where for each episode it resets the environment and, until it is not done, it chooses and executes the action by the current state, tracking the relevant information as the total reward and the way the episode ends.

The agent enjoys of a good grade of exploration during the execution thanks to the "$\epsilon$-greedy" approach. In detail, at each episode an action is randomly chosen with probability $\epsilon$ instead of on the basis of the maximum reward it can return. At each episode, the value of $\epsilon$ becomes $\epsilon = \epsilon \cdot \epsilon_{dec}$ until it does not reach the minimum value $\epsilon_{min}$, where $\epsilon_{dec}$ is the decay factor. It is important to define $\epsilon_{dec} < 1$ for avoiding an increasing value of $\epsilon$ over the episodes.

Once the execution of the function `train()` terminates, it is possible to plot the metrics tracked over the episodes.

### 3.3 Experiments

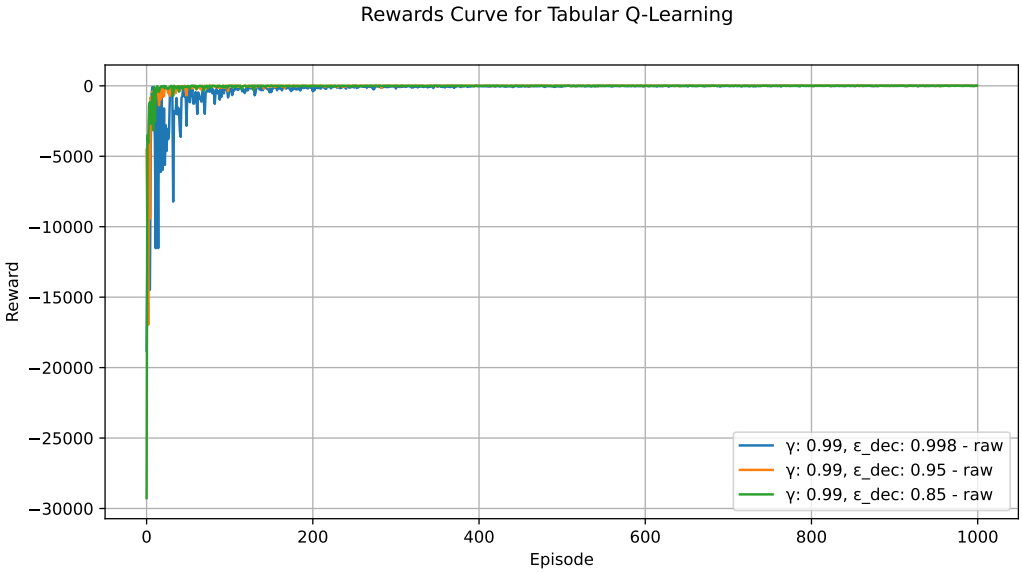
In order to evaluate the performance of the agent, some metrics (such as cumulative rewards and accuracy) are needed to be tracked. The following plots show the results of different runs in which the parameters $\gamma$ and $\epsilon_{dec}$ are changed, while $\epsilon$ and $\epsilon_{min}$ are taken constant to 1.0 and 0.1 respectively. The total amount of episodes for each run is set to 1000, since it is widely enough to get good results.

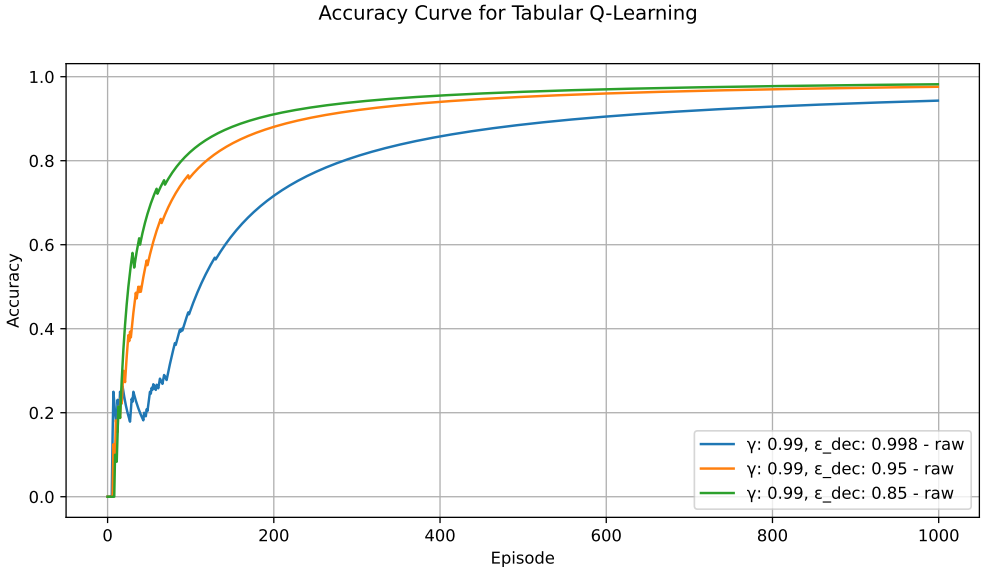
As it was previously said, for the deterministic case $\alpha$ is set to 1, while for the non deterministic it is equal to 0.1.

For each test done by changing the discount factor $\gamma$, it has been done a "zoom" on the cumulative rewards curve plots since it seems hardly indistinguishable the trend of each run. Also for a clear vision, it is adopted a mobile window of 10 episodes for smoothing the original curves.

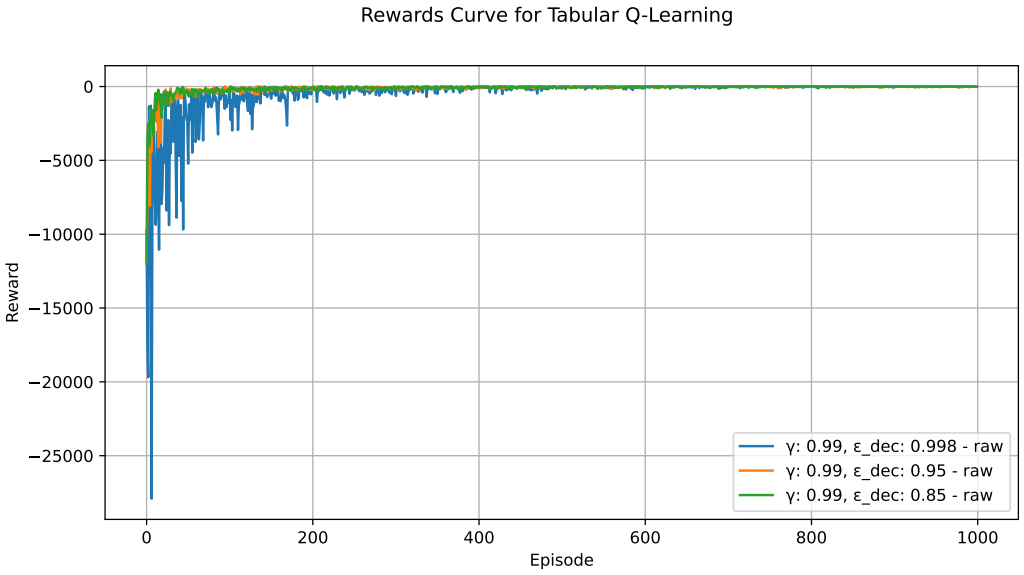
### 3.3.1 $\gamma = 0.99$

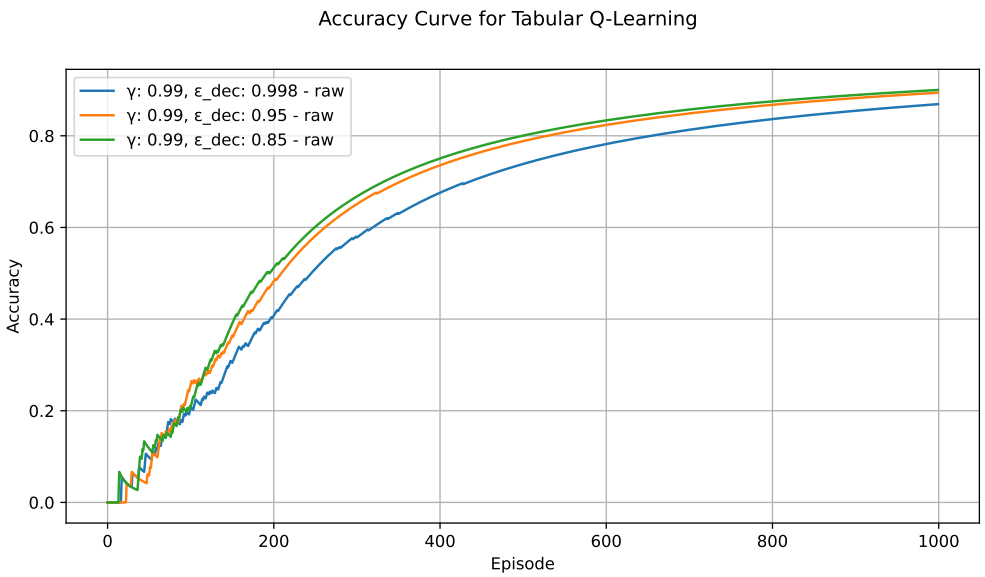
Deterministic plots:




Non deterministic plots:




The deterministic agent finds better result with $\epsilon_{dec}$ equal to 0.85, in terms of accuracy and rewards. It shows a worse behavior the application of $\epsilon_{dec} = 0.998$ until the episode 100.

In the nondeterministic runs, all the agents show a slower trend on reaching optimal value for the accuracy and the rewards, with respect to the deterministic ones.

About the cumulative rewards, it has been obtained the following views:

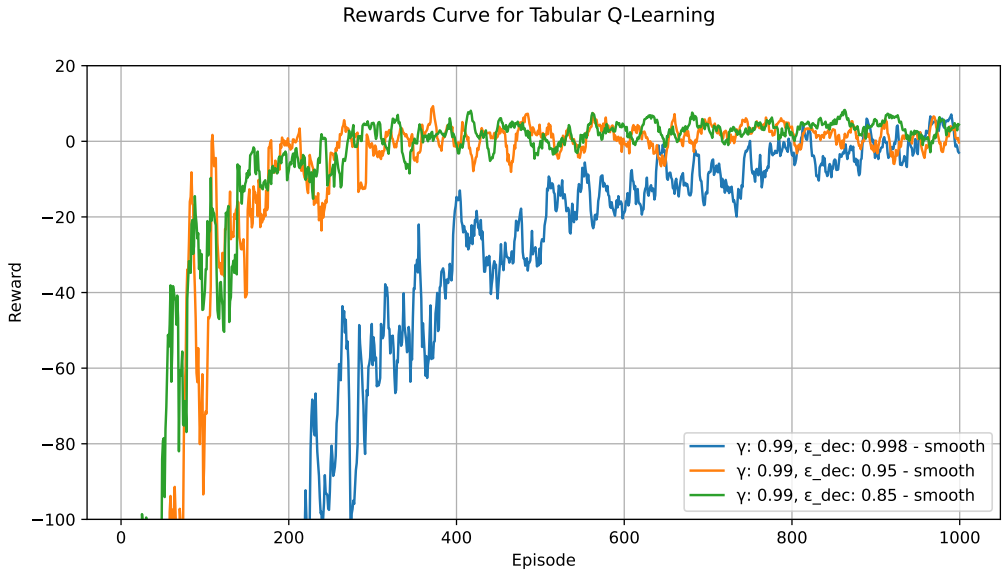



Figure 3: Zoomed view of rewards for deterministic agent

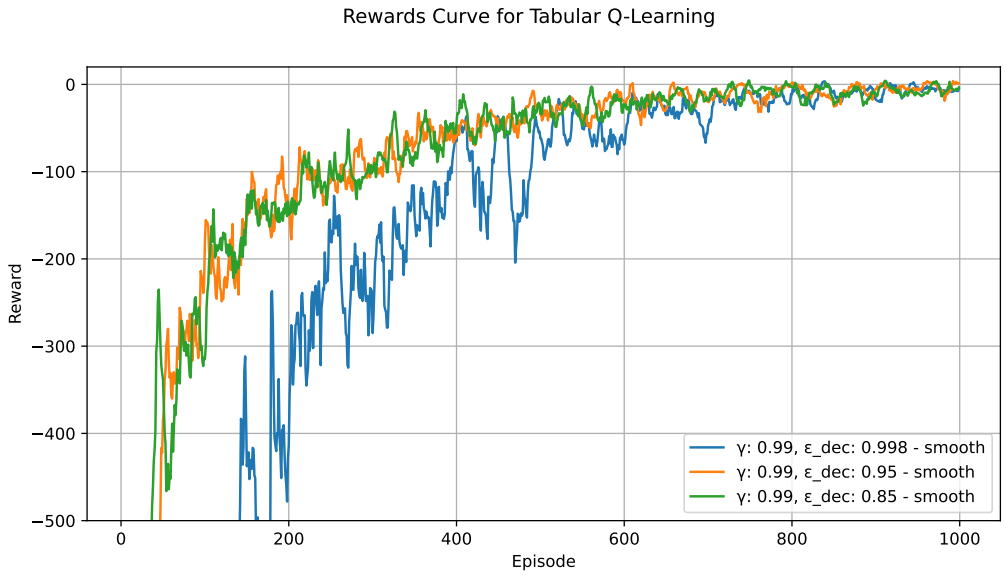
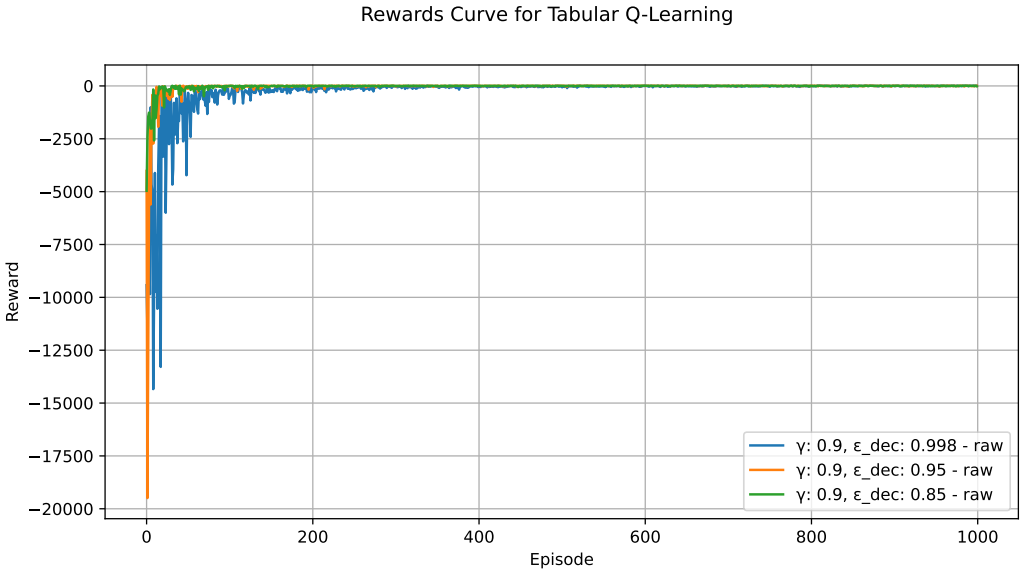


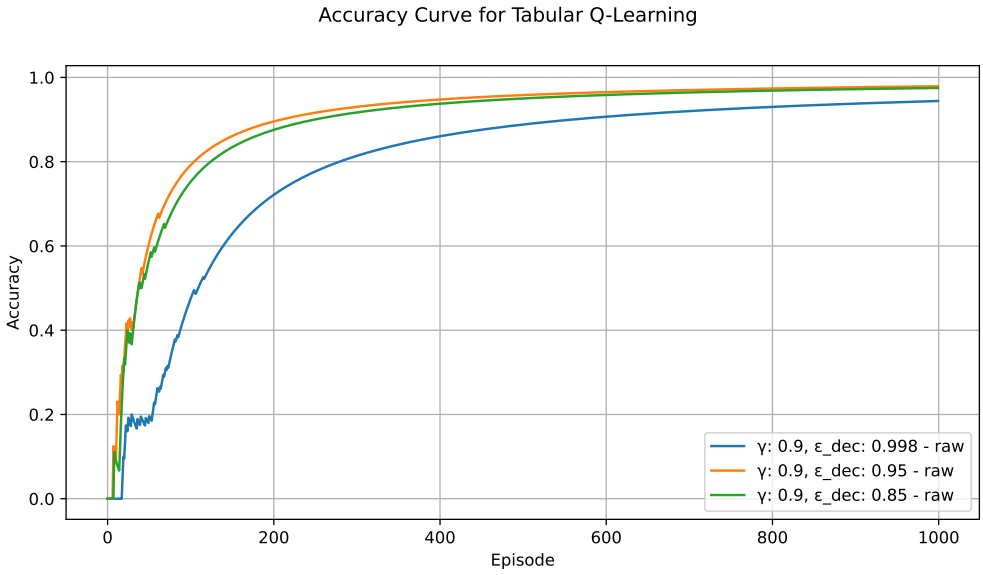

Figure 4: Zoomed view of rewards for nondeterministic agent

From the observations of these plots, it can be seen that the deterministic curves seems to be better than the others, especially by checking the lower bound and a more aggressive growth.

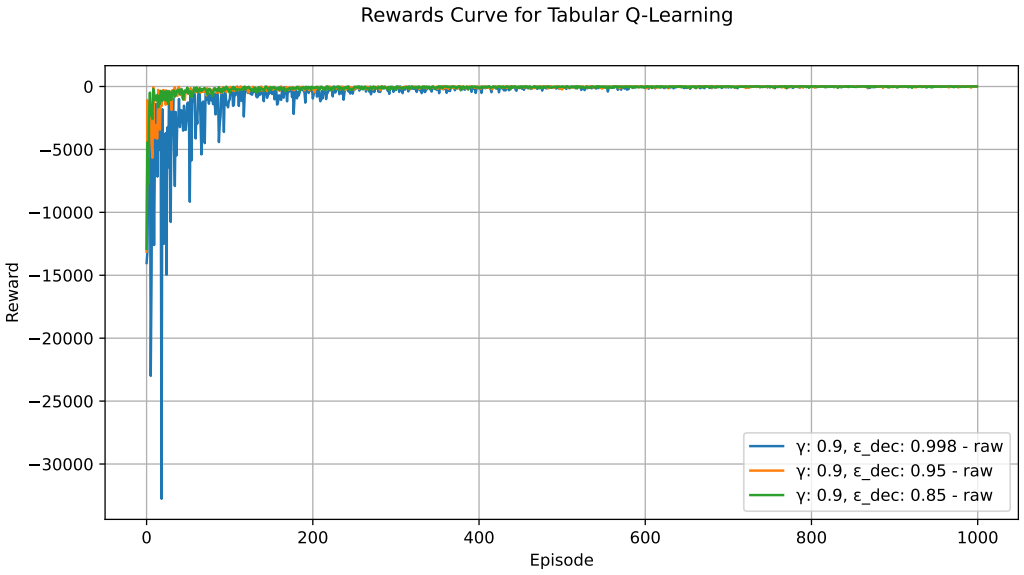
### 3.3.2 $\gamma = 0.9$

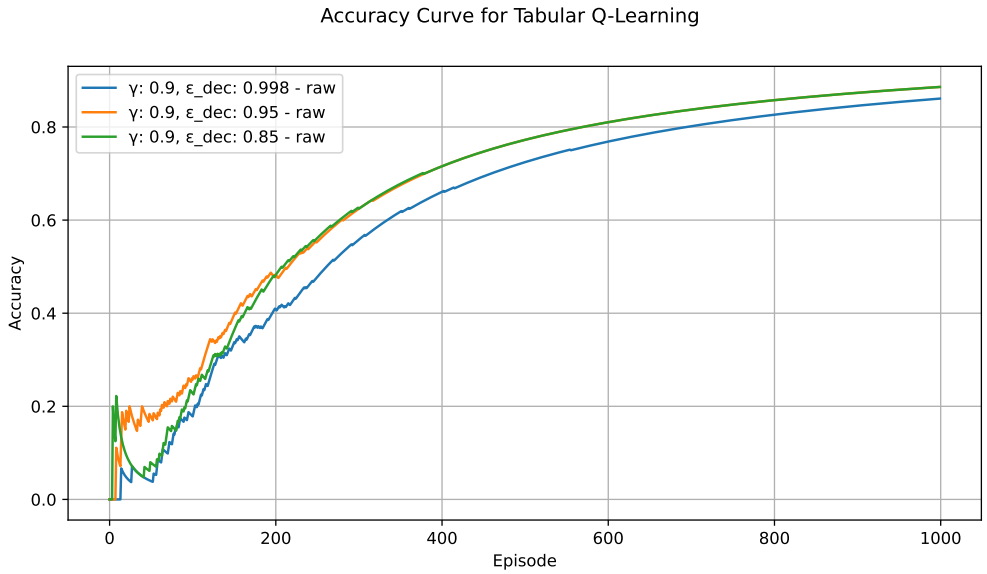
Deterministic plots:




Non deterministic plots:




In both different case, the runs with $\epsilon_{dec} = 0.95$ and $\epsilon_{dec} = 0.85$ show a similar evolution, while the remaining one needs more episodes to reach optimality.

Checking the "zoomed" plot for the rewards there are still no such relevant differences with what observed in the past experiment.

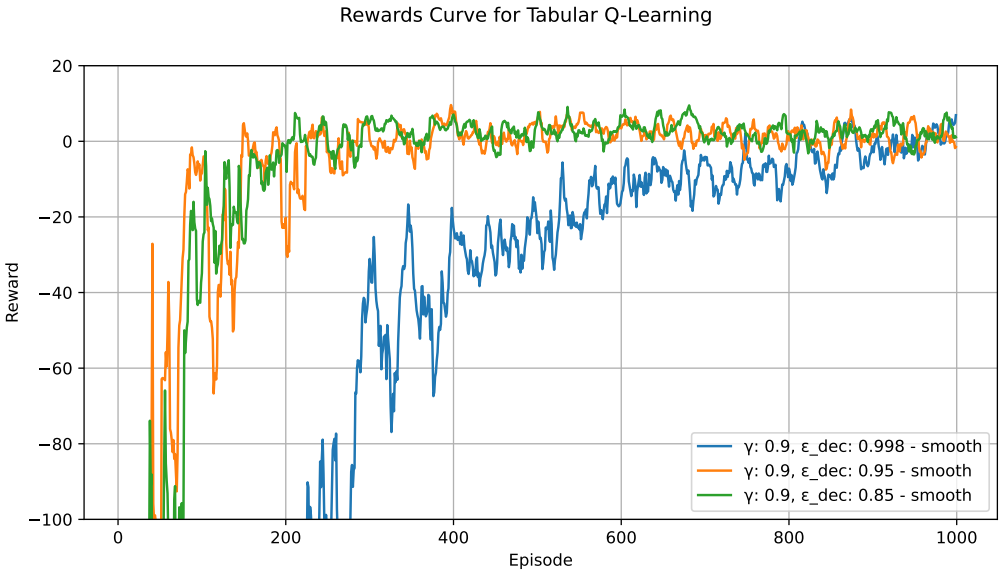



Figure 5: Zoomed view of rewards for deterministic agent

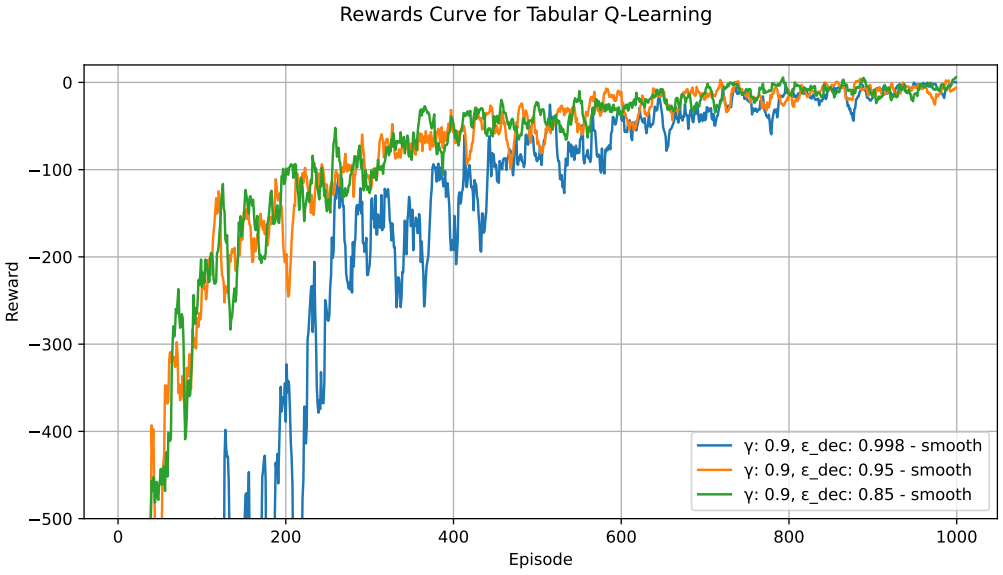
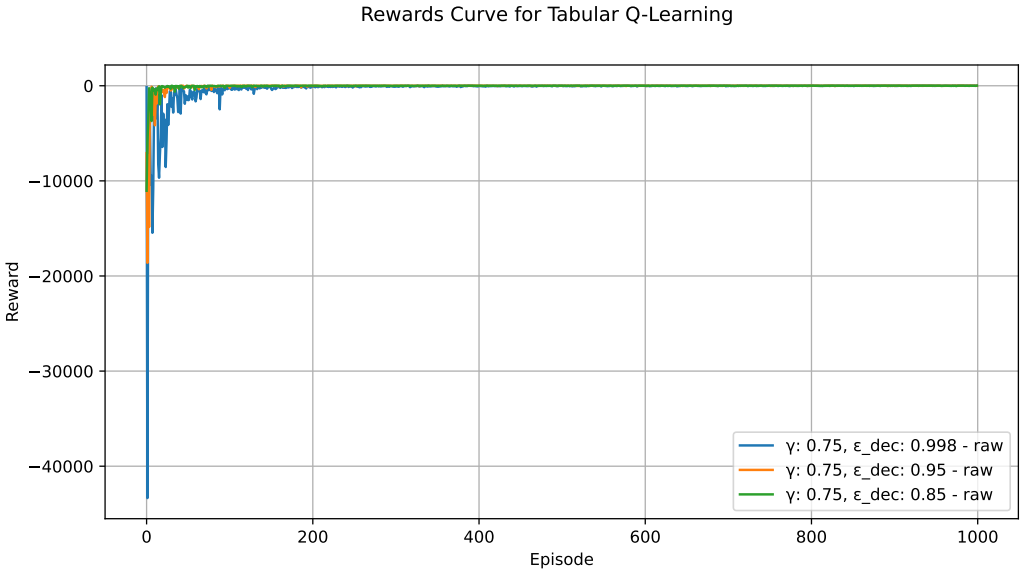
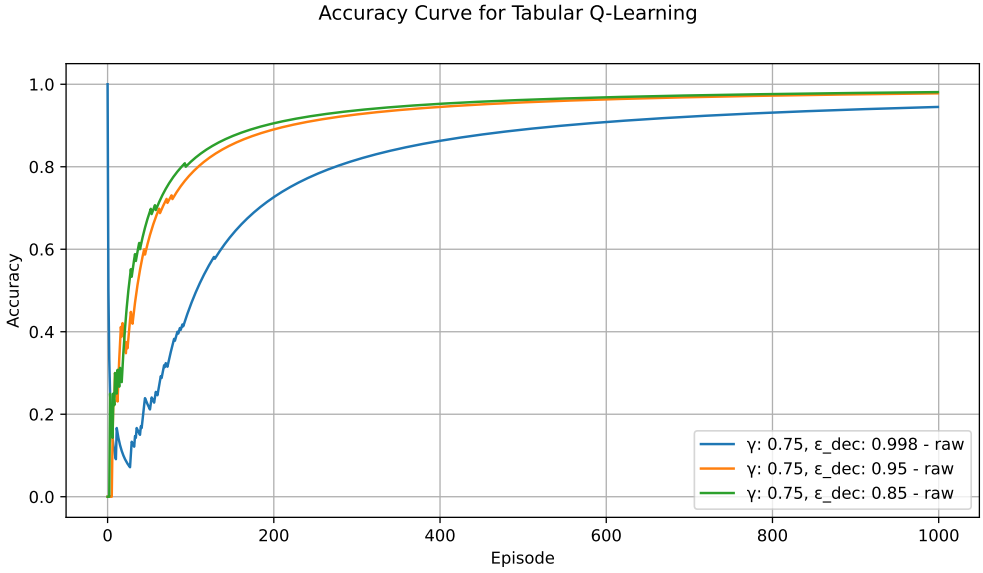



Figure 6: Zoomed view of rewards for nondeterministic agent

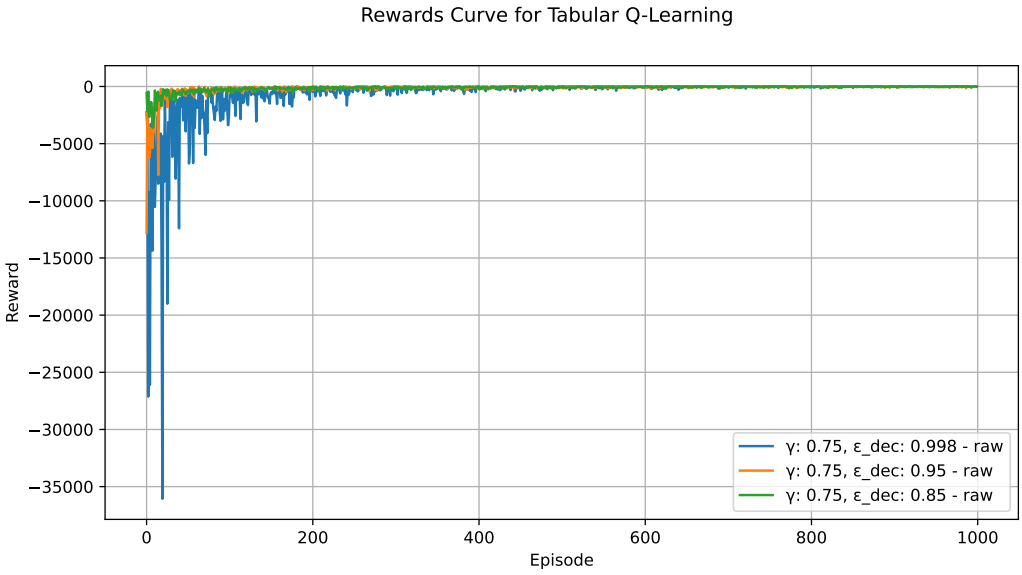
### 3.3.3 $\gamma = 0.75$

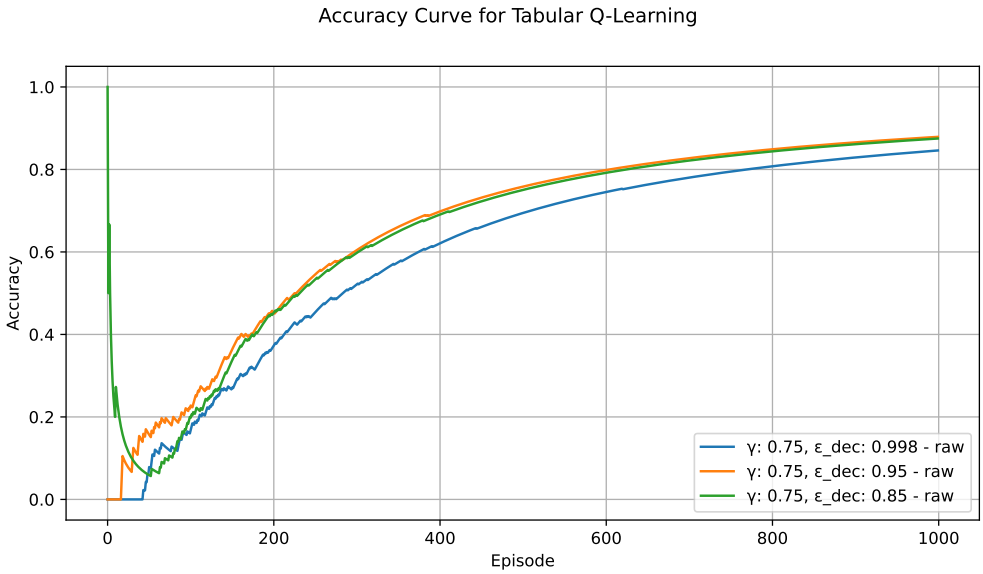
Deterministic plots:




Non deterministic plots:




Checking the graphs, for the deterministic situation can be seen a slightly more stable behavior on the cumulative rewards for the tests done with $\epsilon_{dec} = 0.95$ and $\epsilon_{dec} = 0.85$; this is also reflected on the accuracy. As previously seen, the third value of the decay leads to a slower trend for achieving optimal values.

The same happened for the nondeterministic case but with a more similar accuracy for higher values of $\epsilon_{dec}$.

Here the bigger views of the rewards plots:

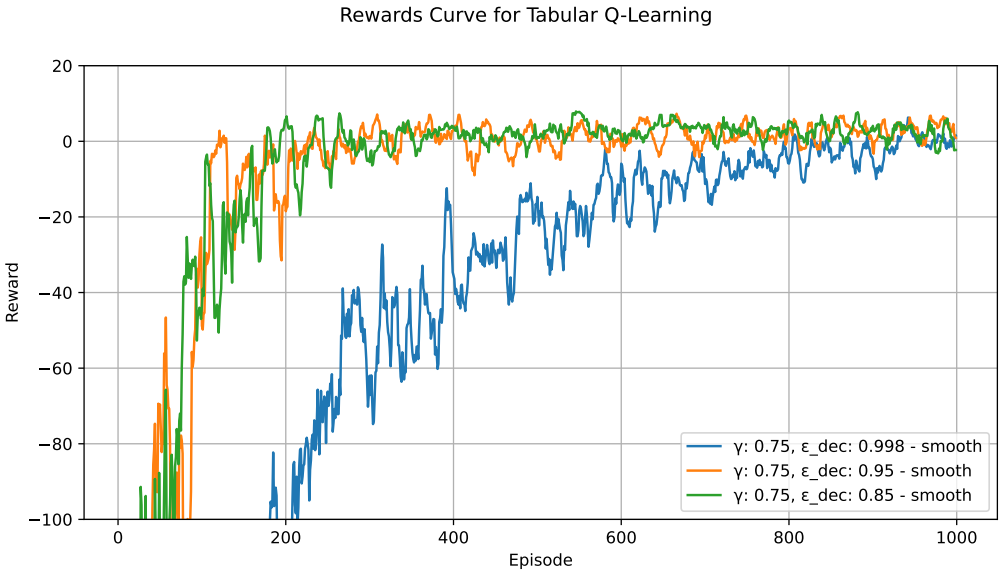



Figure 7: Zoomed view of rewards for deterministic agent

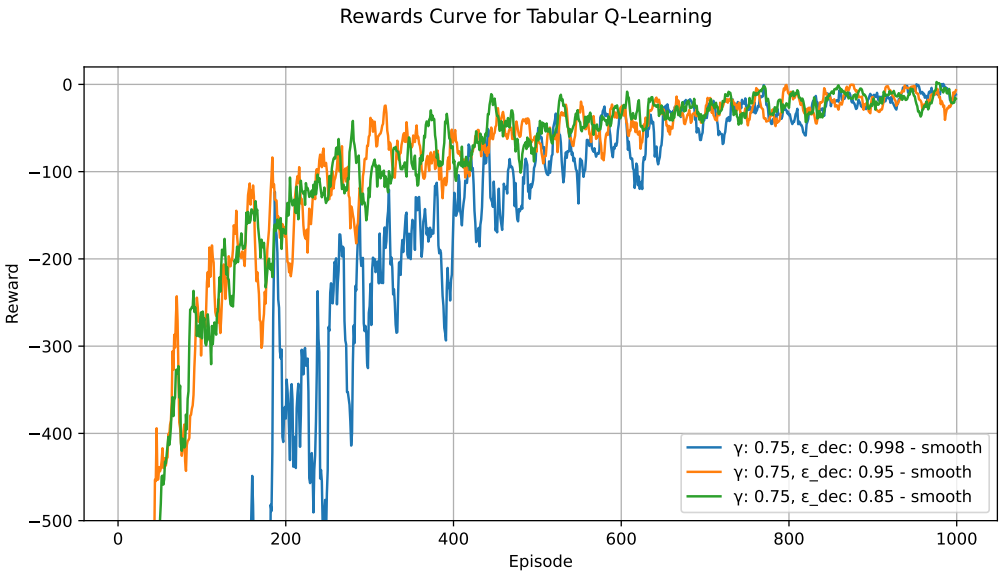
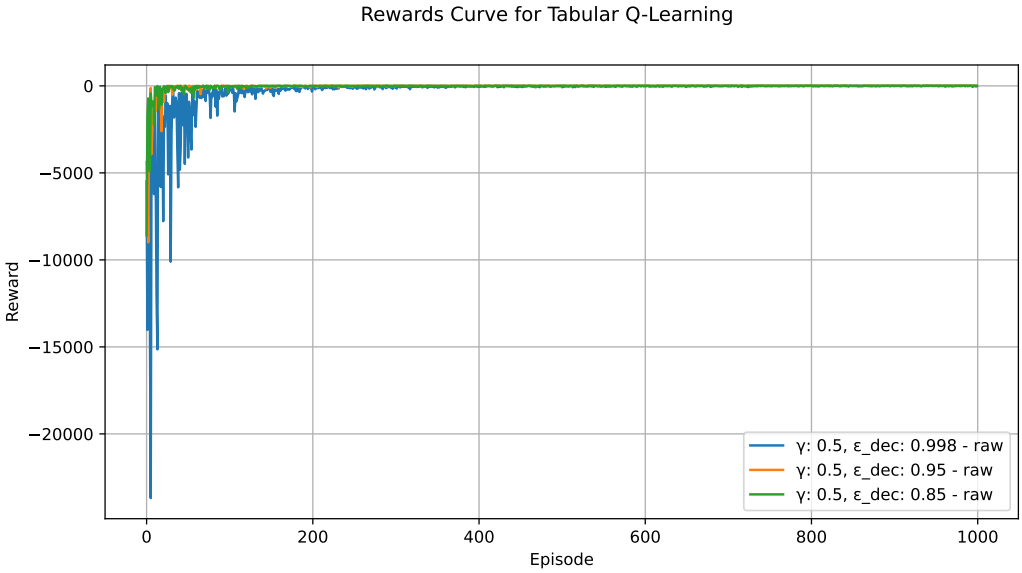


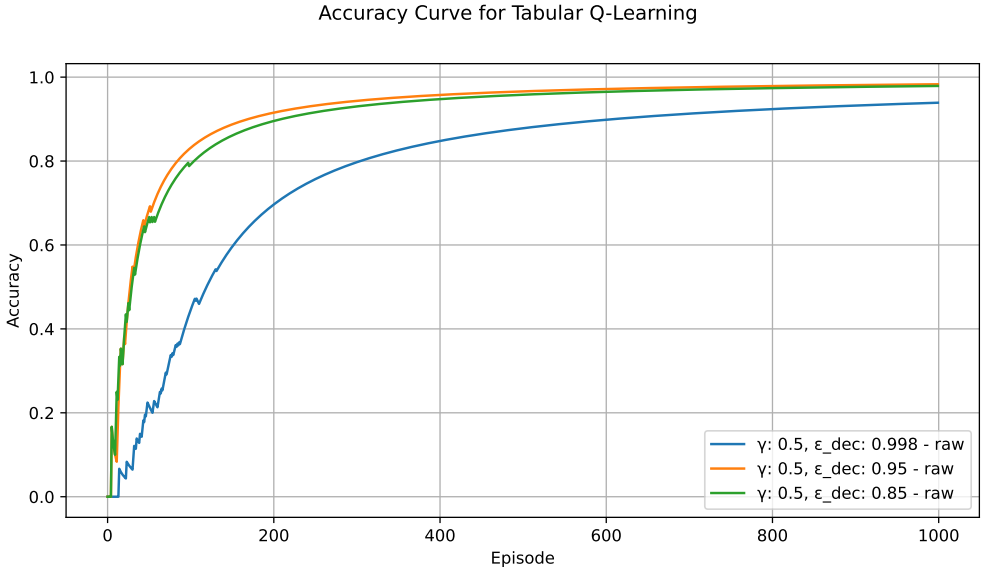

Figure 8: Zoomed view of rewards for nondeterministic agent

About the deterministic agents, it can be evaluated a behavior already seen in other past tests, with no remarkable differences. Instead, on the nondeterminism can be checked wider fluctuations in terms of rewards achieved with $\epsilon_{dec}$ equal to 0.998 .

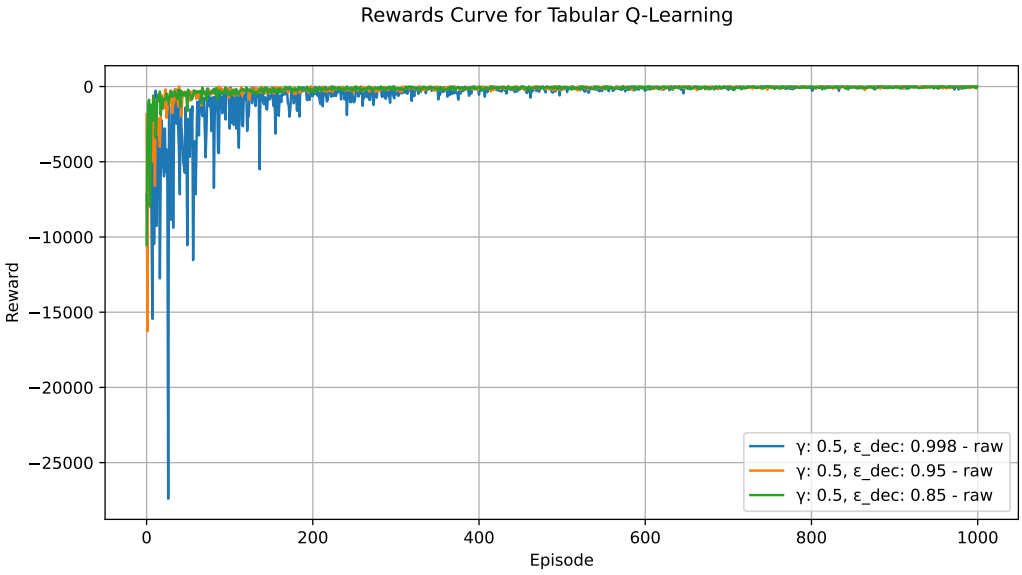
### 3.3.4 $\gamma = 0.5$

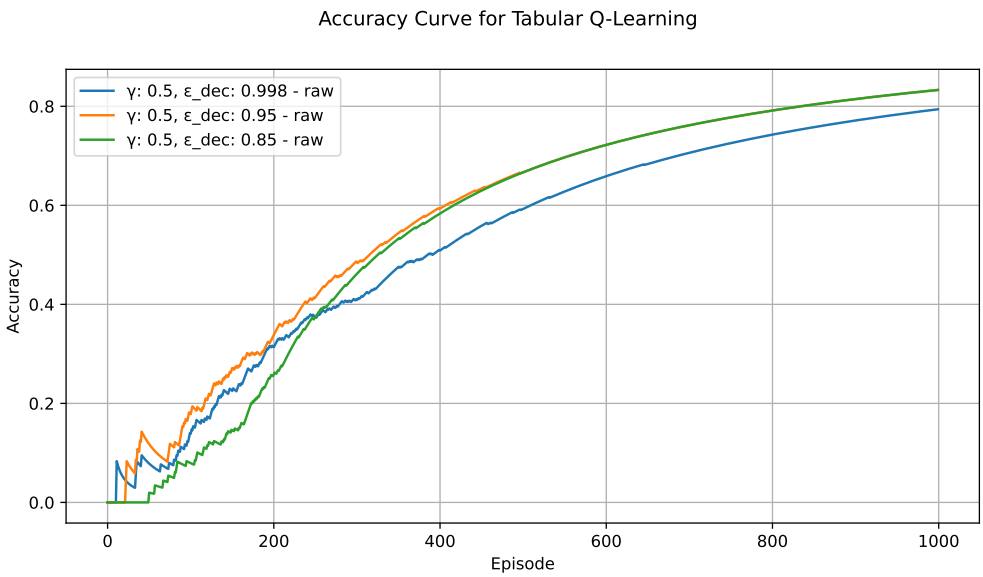
Deterministic plots:




Non deterministic plots:




Even if the deterministic results show no such great differences with what obtained with the previous agents, the nondeterministic ones seem to be worse. In particular, the accuracies start to find a "stable" growth with a higher number of episodes; this can be seen also in the rewards plot where, despite the optimal is reached, that curves have a slower and slightly "toothened" shape.

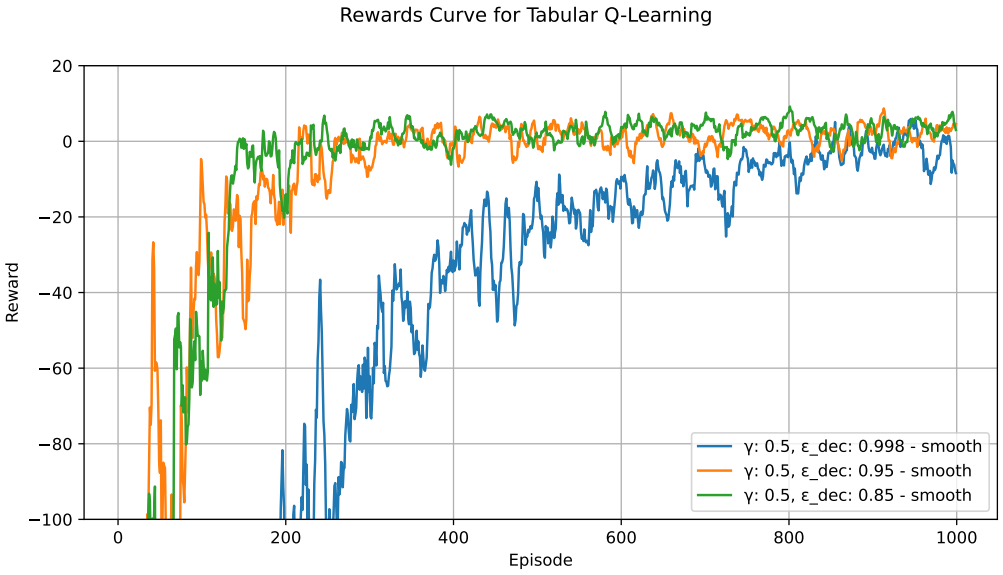



Figure 9: Zoomed view of rewards for deterministic agent

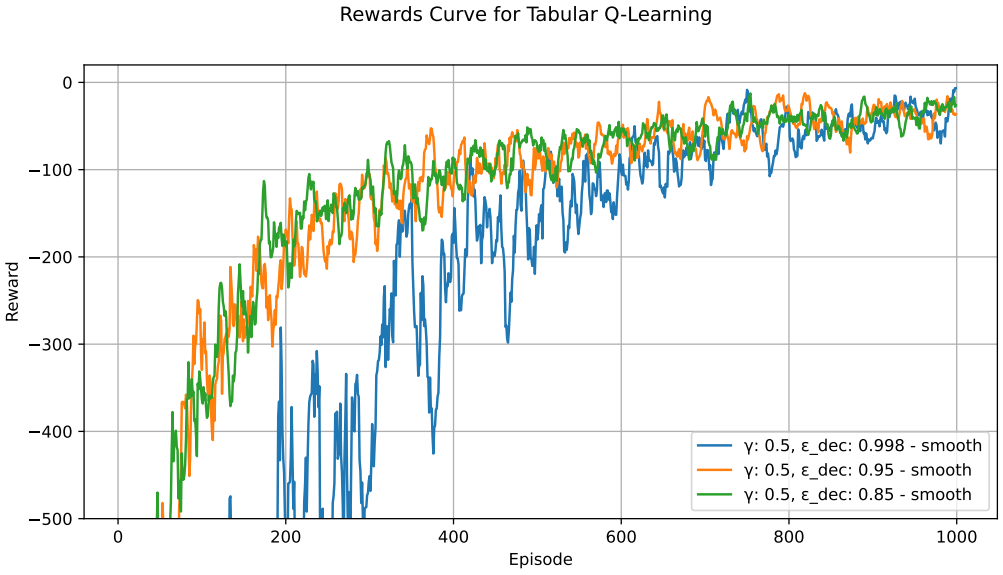



Figure 10: Zoomed view of rewards for nondeterministic agent

## 3.4 Observations

To sum up what has been obtained by the tests for the tabular solution, it emerges that the application of the deterministic Q-function leads to a faster and more stable learning with more consistent results in terms of cumulative rewards and accuracy. Since the nature of this environment is deterministic, using the nondeterministic formula allows to achieve convergence and optimality but with a slower and more fluctuations on the curves of the tracked metrics, as seen in all the runs executed considering $\alpha \neq 1$.

# 4 Second solution: Neural Network

## 4.1 Idea behind the solution

As a more complex computational model for artificial learning, the neural network is based on a similar architecture as the human brain. In particular, it is possible to define nodes as "neurons" placed in so called "layers". In this way, each neuron works as a mathematical function that receives an input and generates an output that can be given as input for other nodes. In its basic configuration, there will be an input and an output layer; for improving performances it might be needed to add one or more "hidden" layers, in charge of elaborating information in order to achieve the learning.

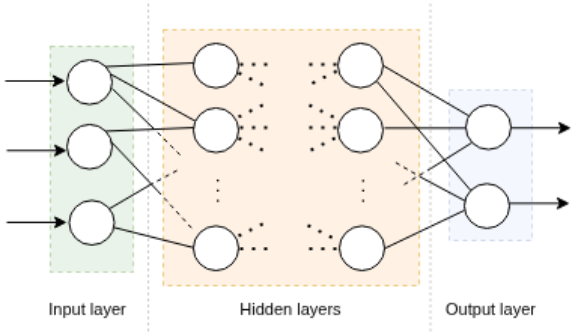



Figure 11: Sketch of the neural network

With respect to the previous solution, the Q-values are obtained through the neural network, given a state as input: this state is processed with linear transformations and activation functions, based on the parameters of the network. The output is used for the computation of the target value, which is useful for determining the loss of the current step and performing the update of the parameters.

Also for this solution, the Q function adopted is the following:

$$Q(s,a) = (1-\alpha)\cdot Q(s,a) + \alpha(r + \gamma \cdot \underset{a' \in A}{max}(Q(s',a')))$$

taking $\alpha = 1$ for the deterministic runs and $\alpha \in (0,1)$ for the nondeterministic ones.

The agent works directly with a neural network to elaborate data for solving the environment.

## 4.2 Implementation

In this solution there are some classes: `Agent` for the agent, `ReplayBuffer` for the replay buffer and three different ones for the neural networks.

For storing the information about the experiences, it is required a `Replay Buffer`: it consists on a data structure like a buffer where is possible to push elements and pick some of them randomly.

For the neural network classes, there have been defined three different classes `Lx_QNet` that implement a network with an input layer, $x-2$ hidden layers and an output layer. The parameters for the classes are `state_dim` and `action_dim`, representing the dimension of the state returned by the environment and the size of the structure containing the possible actions executable. The three different neural networks are developed with `L3_QNet`, `L4_QNet` and `L5_QNet` respectively for 1, 2 and 3 hidden layers other than input and output ones.

The main core of these classes are the functions `nn.Linear()` and `torch.relu()`, directly taken from the library Pytorch and its relative module for neural networks:

- `nn.Linear()` is the module that generates a layer with `input_dim` and `output_dim`, performing a linear transformation, following the equation $y = xW^T + b$, on the given input;
- `torch.relu()` is the function that activates the rectified linear unit, it puts to 0 all the negative values and it keeps as they are only the positives.

The flow of the data through the network follows a sequential behavior for giving an output by the previously given input: this operation is performed by the `Lx_QNet.forward()` function.

About `ReplayBuffer`, the implementation follows few functions for pushing elements and randomly taking a minibatch of elements, with an initialization that sets the size of the buffer by a given parameter.

The class `Agent` is mainly characterized by the following functions:

- `select_action()`, as for the tabular it uses a "$\epsilon$-greedy" approach for choosing the next action to perform, where with probability $1 - \epsilon$ is taken an action by reasoning on the state in tensor form (obtained by `torch.tensor()` and `torch.nn.functional.one_hot()` for ensuring uniqueness);
- `replay()`, it mainly consists on the following steps:
  1. extraction of information about states, actions, rewards and next states from the elements of the minibatch for generating lists;
  2. conversion of the previous data structures in tensors through `torch.tensor()`, for states and next states it is used `F.one_hot` for ensuring uniqueness during the computation;
  3. computation of actual and future Q-values using the network and the tensor of the actions;
  4. evaluation of the target through the Q-function reported above;
  5. calculation of the loss using `F.mse_loss()`, so considering it as mean squared error;
  6. backpropagation of the information given by the loss for computing the gradients and updating the parameters of the network, thanks to `backward()` and `optimizer.step()`

The agent, when possible, by the function `to(self.device)` loads directly the model into the dedicated graphical processing unit for better performances. Another technical adoption for improving metrics is the optimizer `Adam`, that uses a mobile window of gradients and squared gradients in order to change automatically the learning rate for each parameter and the momentum for optimizing the descent of the gradient.

The `Train()` function is the one that performs all the computations. It takes as input the number of episodes, $\gamma$, $\epsilon$, $\alpha$, $\epsilon_{dec}$, $\epsilon_{min}$, learning rate of the learning algorithm `Adam` and $\alpha$, the learning rate of the Q function. First of all, it creates the three different agents having the neural networks with 3, 4 and 5 total layers respectively. For each of them and for each episode, it resets the environment and, while the episode is not done, chooses the action $a$ by the current state and it executes $a$. Later that, the relevant information returned by the environment are pushed into the replay buffer and, then, the `replay()` function is invoked. The value of $\epsilon$ is updated at the end of the episode. Each time an agent terminates its execution, the plots referred to accuracy, loss and rewards will be printed in their dedicated files.

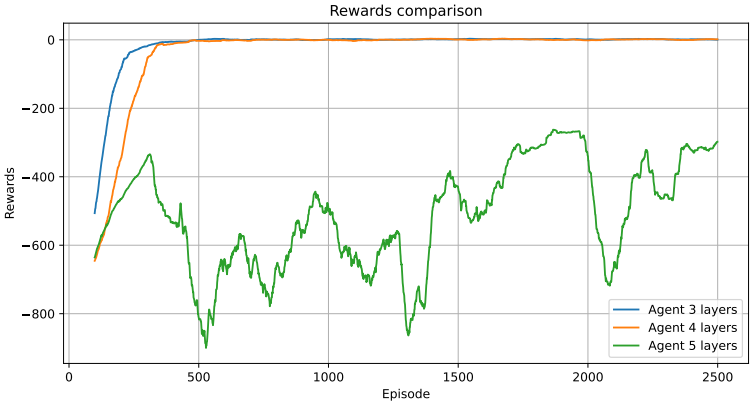
## 4.3 Experiments

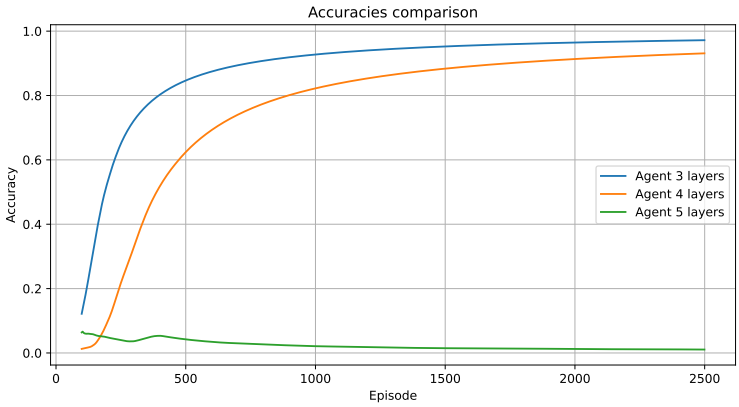
For the three agents, the tests done are focused on changing of the dimension of the `Replay Buffer` for the deterministic case and the nondeterministic. The other parameters of the agents have fixed values: $\gamma = 0.99$, $\epsilon = 1$, $\alpha = 0.1$ for the nondeterministic, $\epsilon_{dec} = 0.995$, $\epsilon_{min} = 0.1$ . The total number of episodes for each run is 2500.

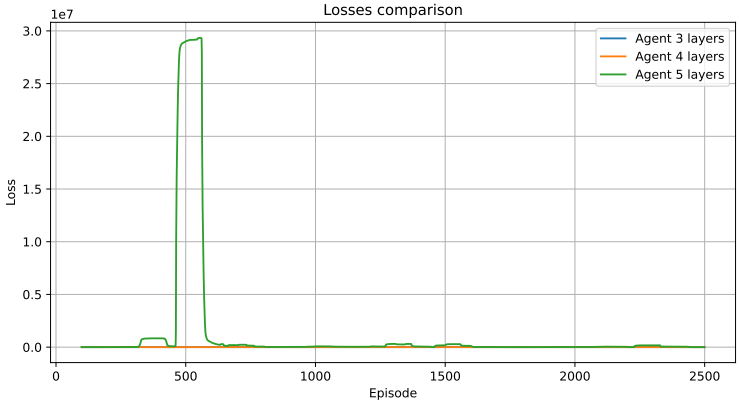
The Adam algorithm has a learning rate equal to 0.001, as suggested by the official documentation of PyTorch. The number of neurons for each hidden layer is set to 64.

### 4.3.1 Buffer size equal to 5000

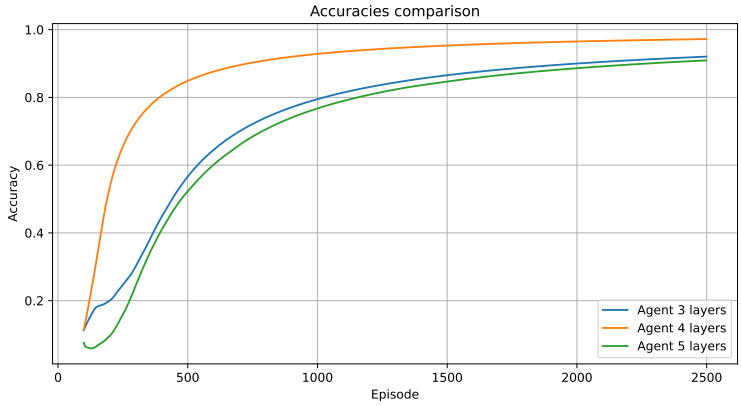
Deterministic

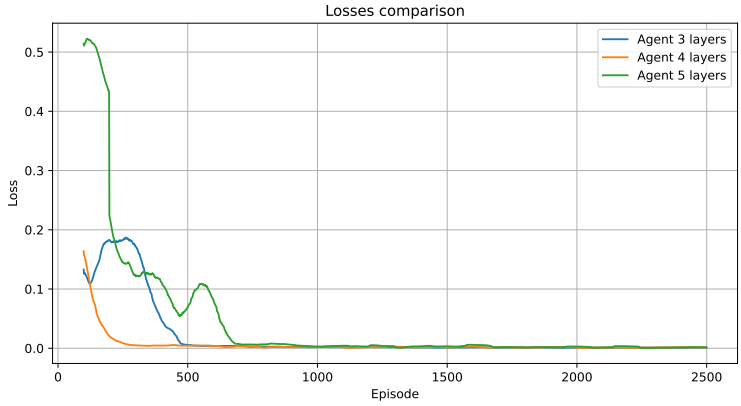



Nondeterministic:

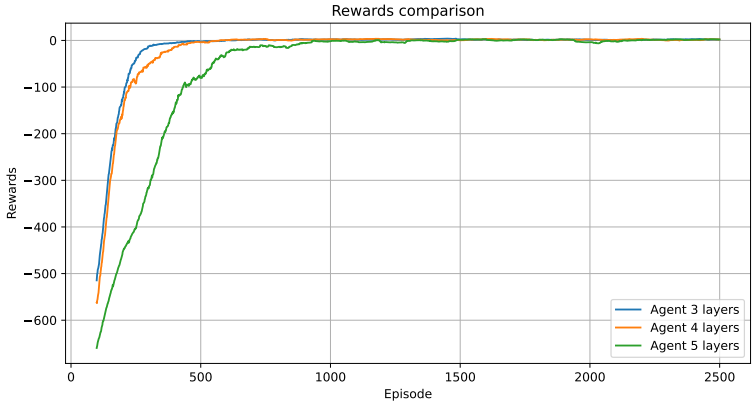



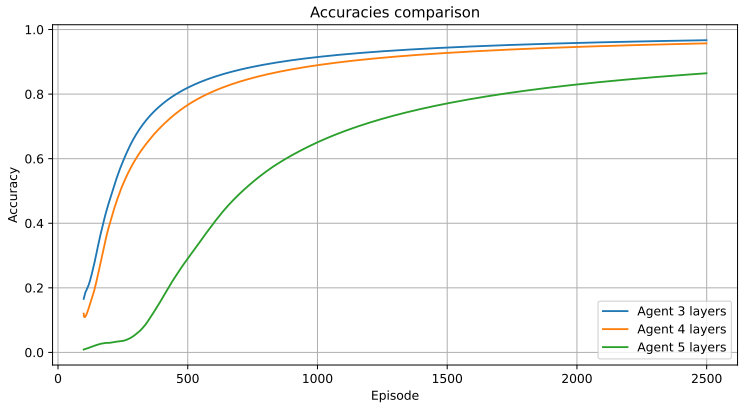



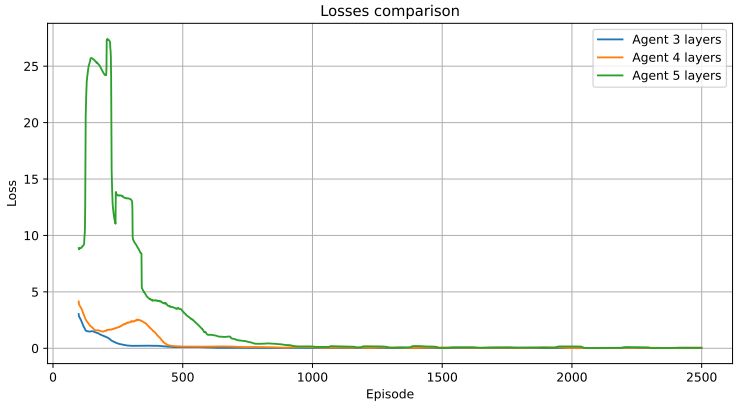

**Observation on the test** Both the two applications of $\alpha$ led to acceptable performances for all the agent; the only exception is represented by the deterministic neural network with 3 hidden layers (the so called "5 layers agent"), where by just looking at the rewards plot can be observed an irregular trend that does not reach optimal values.

### 4.3.2 Buffer size equal to 10000

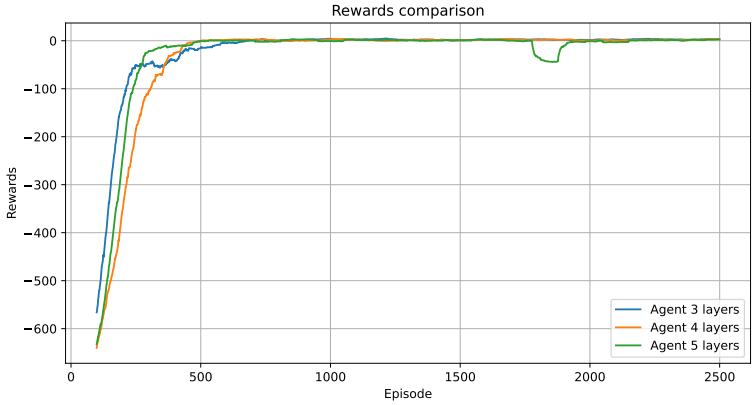
Deterministic

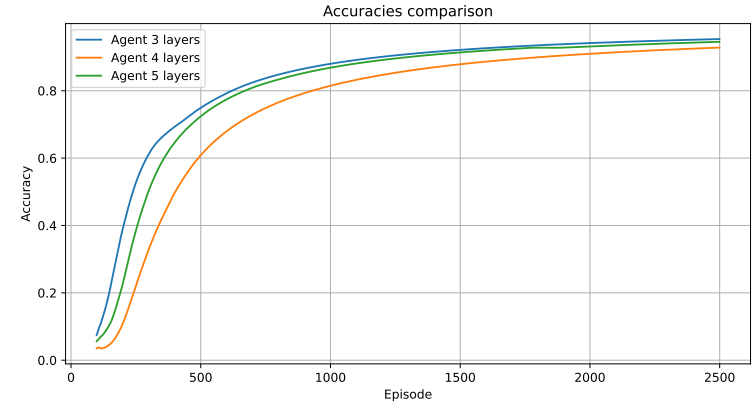


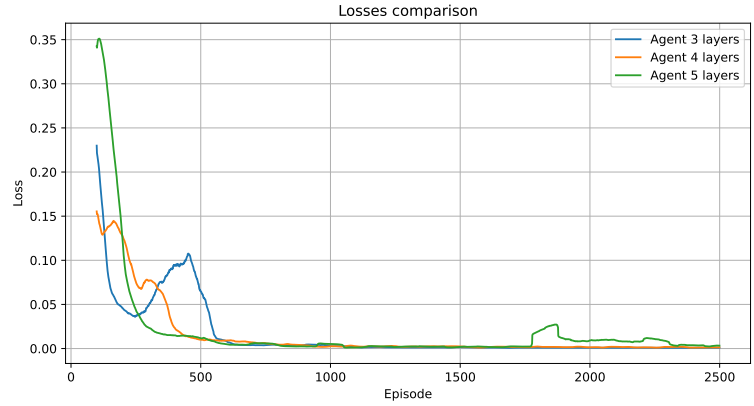




Nondeterministic:



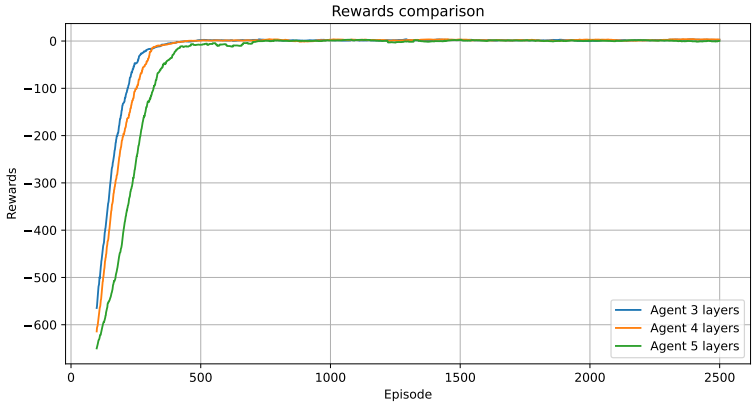



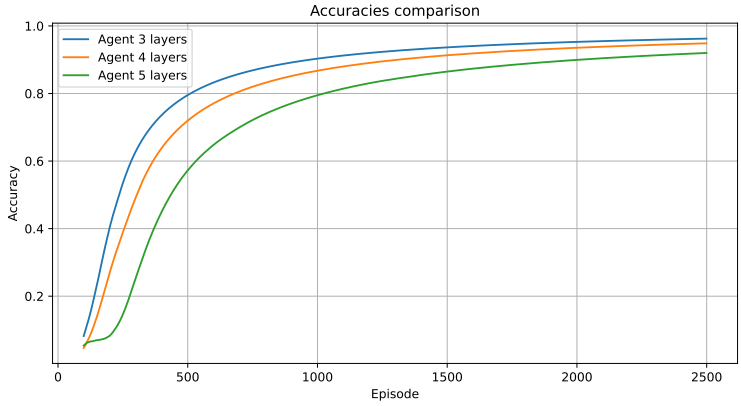

**Observation on the test** With a size of 10000 elements, the Replay Buffer seems to be reasonably large for allowing all the agents to find the optimal policy, even if the 5 layer agent in the nondeterministic case, between the episodes 1500 and 2000, shows an anomaly.

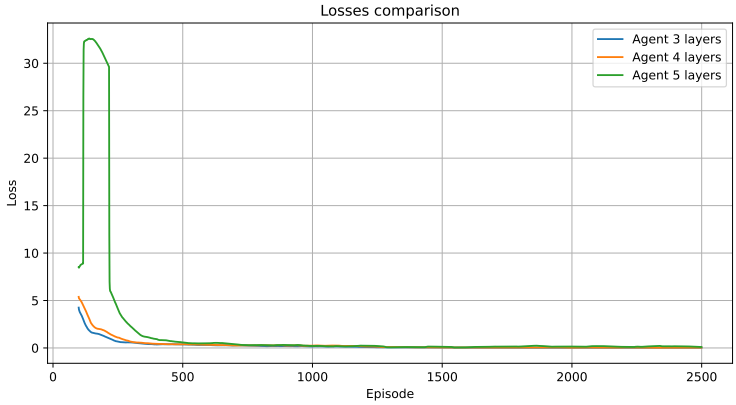
Here it is reported the "zoomed" rewards plot for observing better the problem reported above:


### 4.3.3 Buffer size equal to 20000

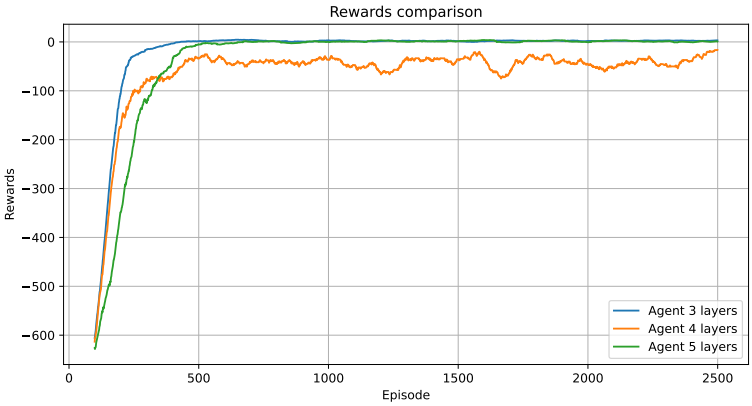
Deterministic

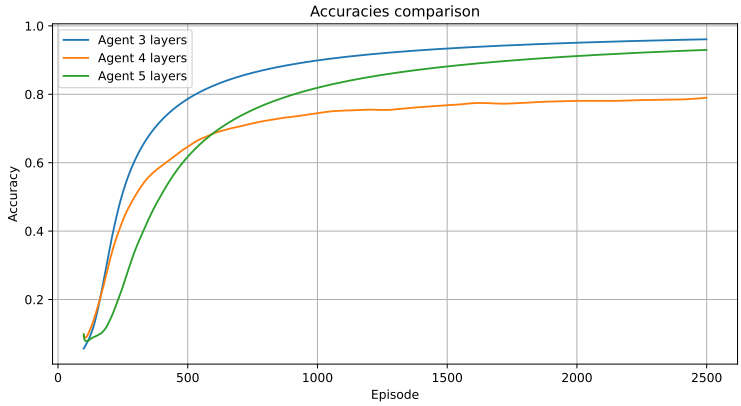


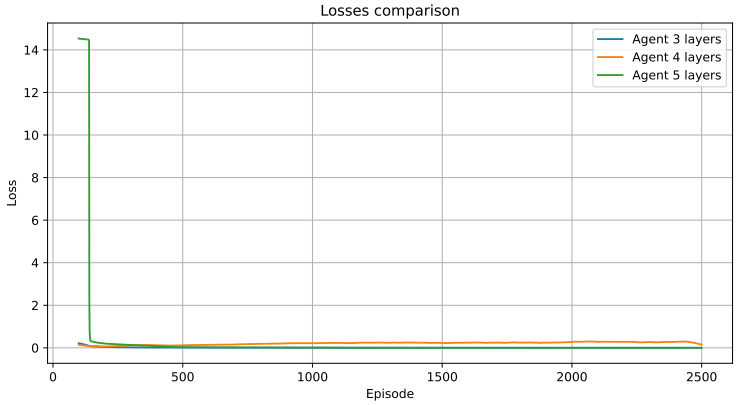






Nondeterministic:



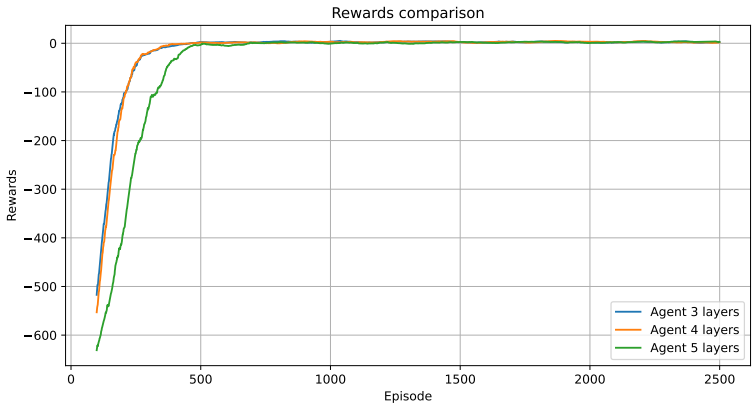



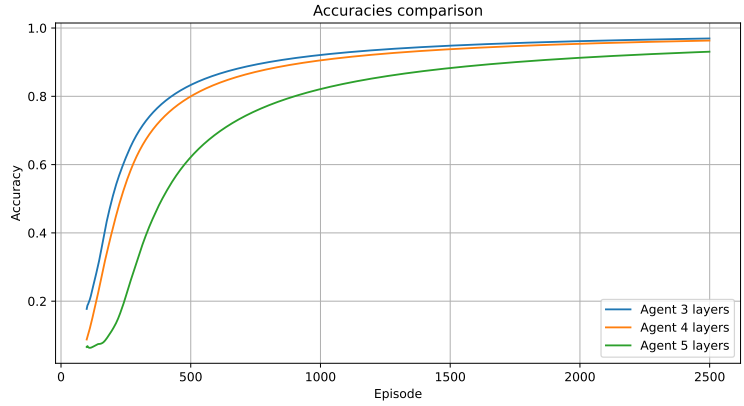


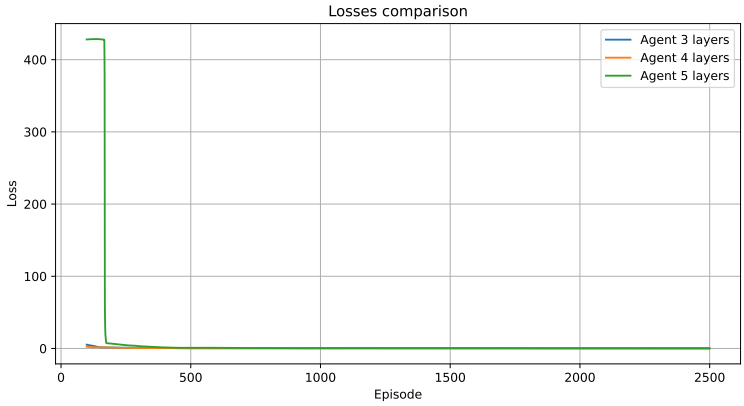

**Observation on the test** The agents in the deterministic scenario seem a very good behavior on finding a good strategy for solving the environment, reflected on the available metrics. Also in the nondeterministic situation the agents work well, even if the 4 layers network finds stability on value around -50 for the rewards (as it is possible to see on the zoomed plot) and 0.8 for the accuracy:


### 4.3.4 Buffer size equal to 100000

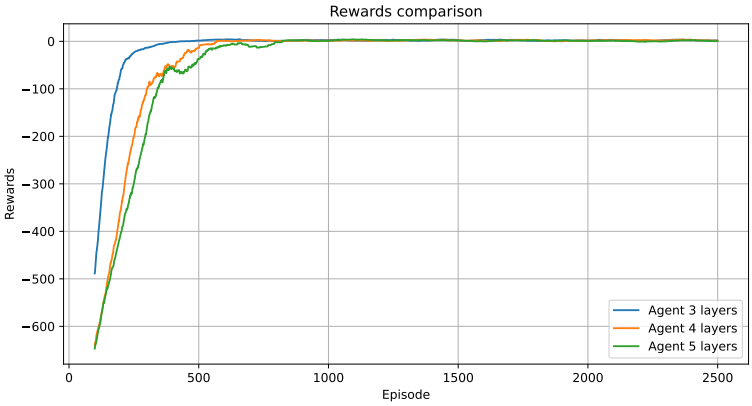
Deterministic

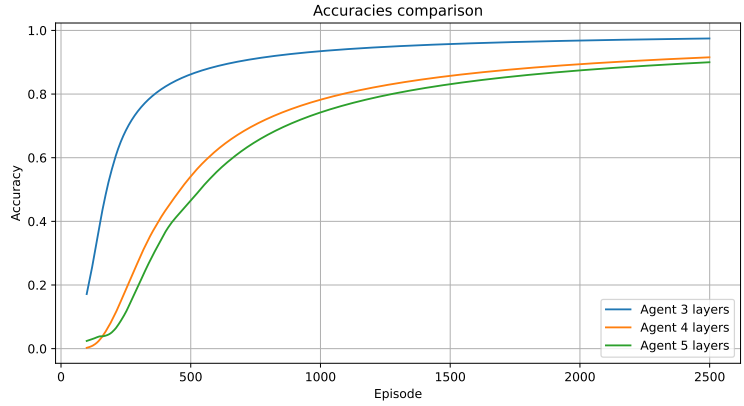


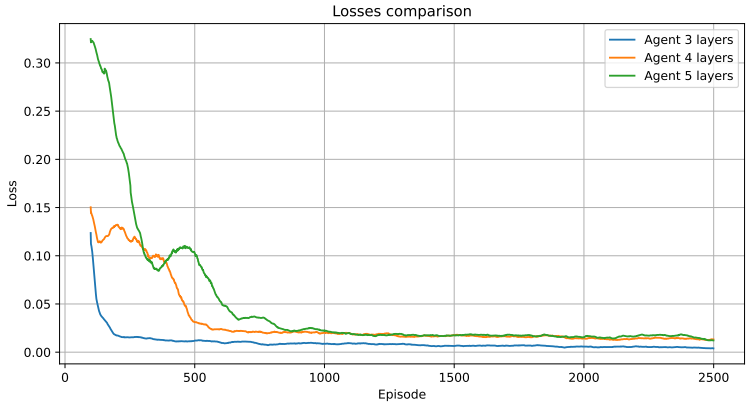



Nondeterministic:






**Observation on the test** A buffer with a size of 100000 brings the best results for this environment. As theory suggests, the application of the nondeterministic formula to a deterministic environment implies a "slower and more gradual" convergence to the optimal values.

## 4.4 General observations

Taking a comparison between Tabular solution and DQN solution it comes out that the DQN requires more episodes for optimizing the tracked metrics, both for deterministic and for nondeterministic case. This can be explained from the facts that the neural networks are more complex then a "simple" table, and from the discrete-state-nature of the environment that leads to easier presentation of the Q-values.

# 5 Third solution: Neural Network with Target Network

## 5.1 Idea behind the solution

Starting from the previous idea of neural network, in this solution is adopted another network called "Target". This should allow to make the learning more stable and precise thanks to a variation of the Q-values, used for obtaining the target and in the loss for optimizing the parameters of the network, each a fixed number of episodes. In fact, the Q-values for the target are taken from this second network while the ones for the current state are obtained through the main one (updated after each action). The application of that concept may not cause a faster convergence to optimal values, but it has the goal to make the agent able to find a better estimation along the time.

## 5.2 Implementation

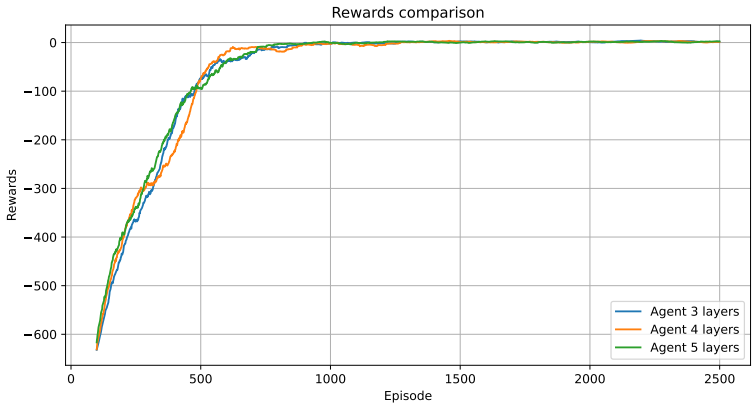
With respect to the previous implementation, there are only few differences. The first one relies on the class `Agent`, in particular inside the functions `init()` and `replay()`: instead of using only one network, there are the `q_network` and the `target_network`, used for retrieving Q-values of the current state and the ones related to the next state respectively. The Q-Network is updated with the same frequency of the previous case, so after each invocation of the `replay()` function, meanwhile the Target network is updated each 10 episodes on the basis of the values stored inside Q-Network.

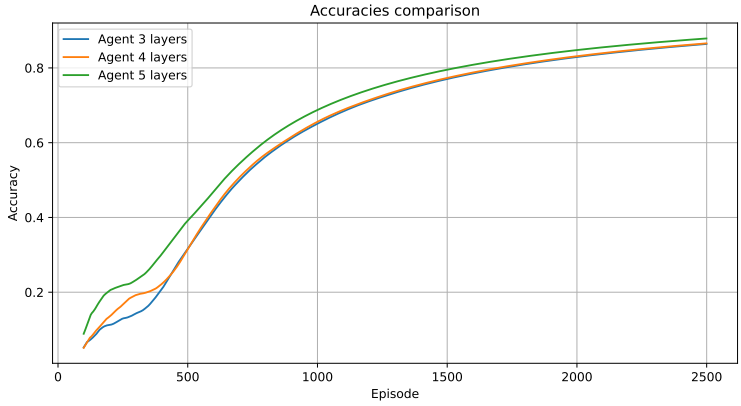
## 5.3 Experiments

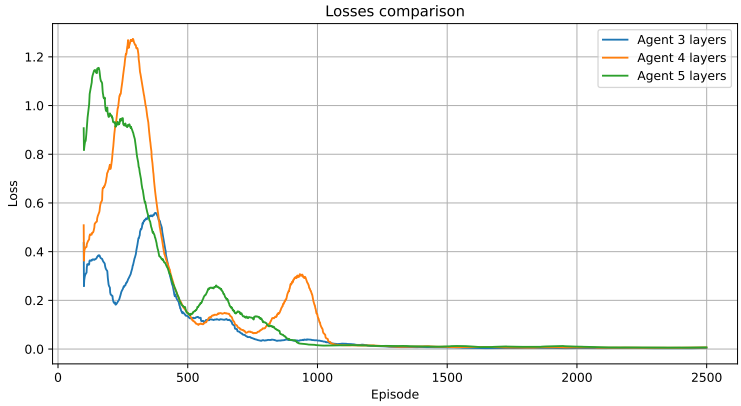
The tests done for this solution follows the same schema described in the "single" network implementation. So, for refreshing what has been set constant: $\gamma = 0.99$, $\epsilon = 1$, $\alpha = 0.1$ for the nondeterministic, $\epsilon_{dec} = 0.995$, $\epsilon_{min} = 0.1$ and 2500 episodes for each test.

### 5.3.1 Buffer size equal to 5000

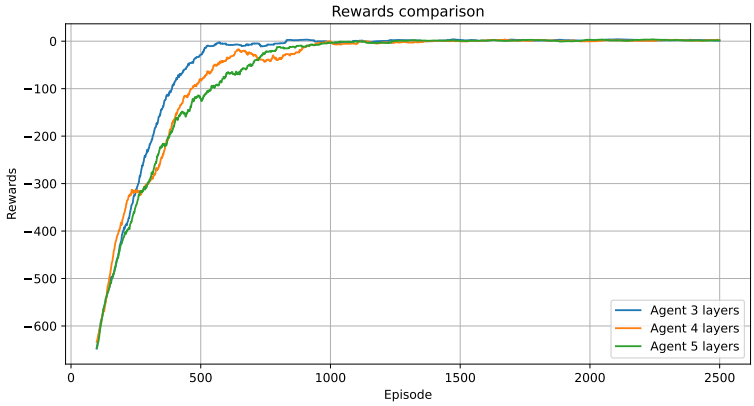
Deterministic

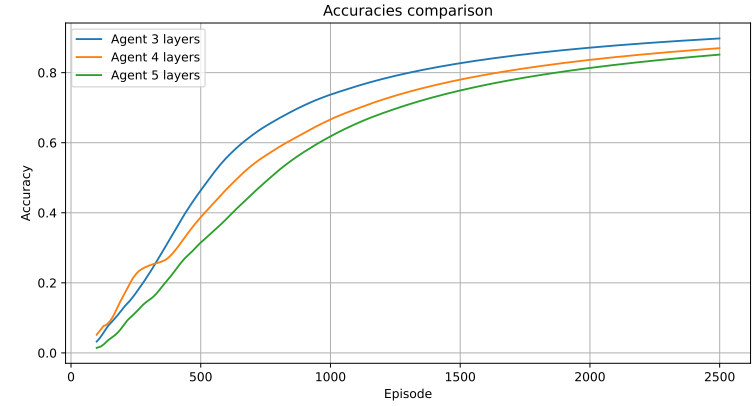


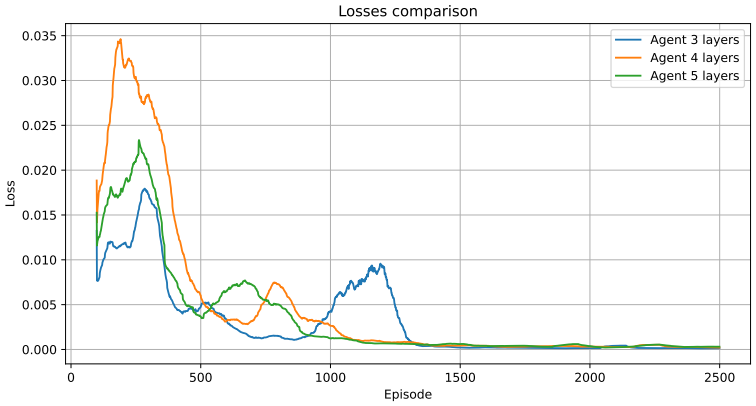



Nondeterministic:



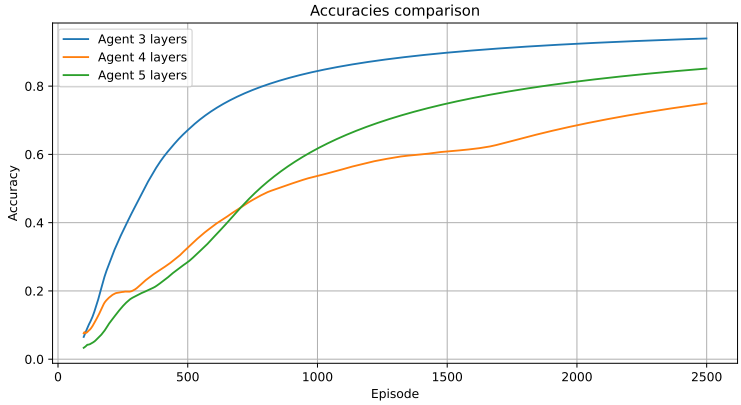


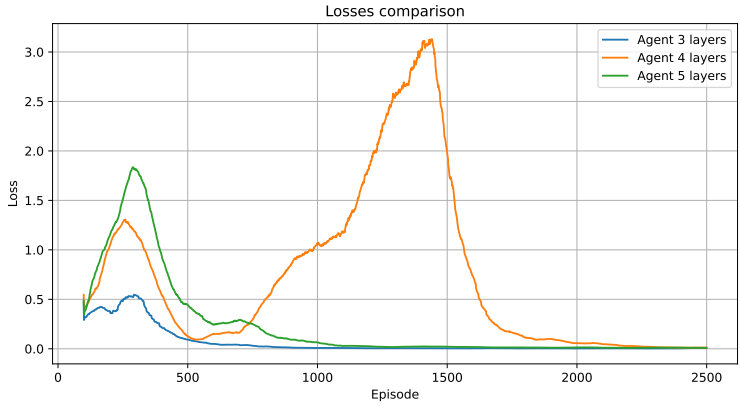




**Observation on the test** All the runs are pretty good. The optimal policy is found gradually, despite the presence of small oscillations in the learning phase. As wanted, the trend of the losses is not uniform until they start to tend at 0 value without any particularly relevant "spikes".

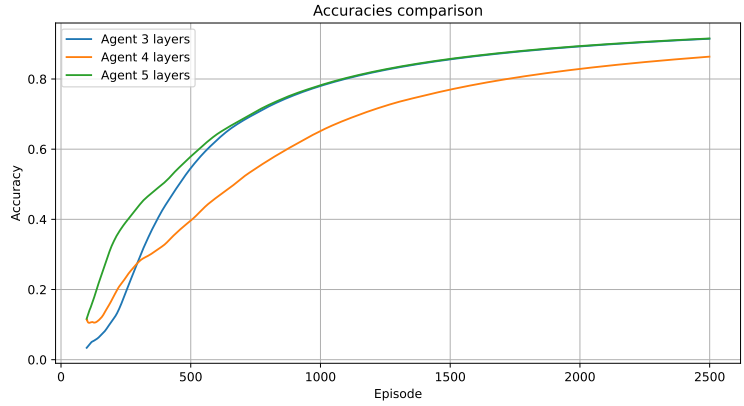
### 5.3.2 Buffer size equal to 10000

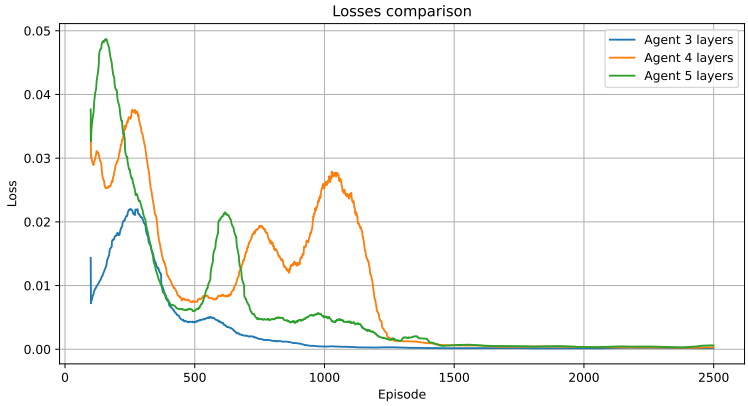
Deterministic




Nondeterministic:

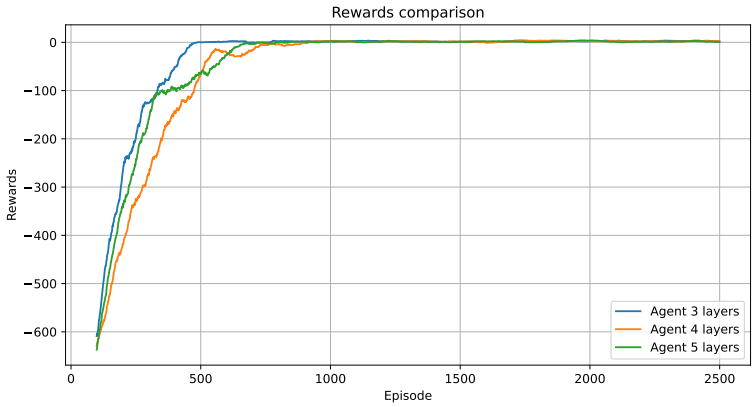


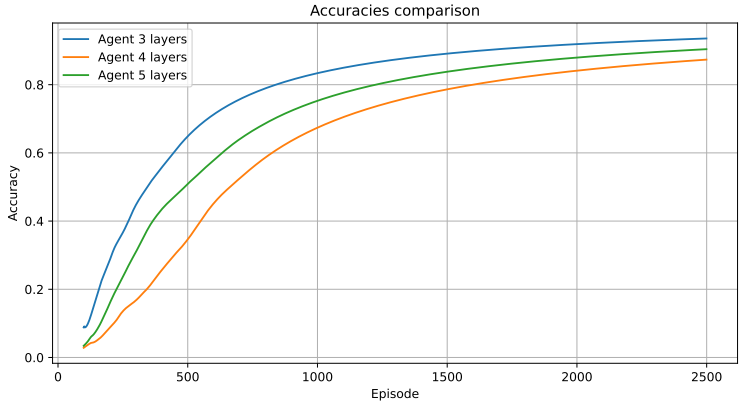



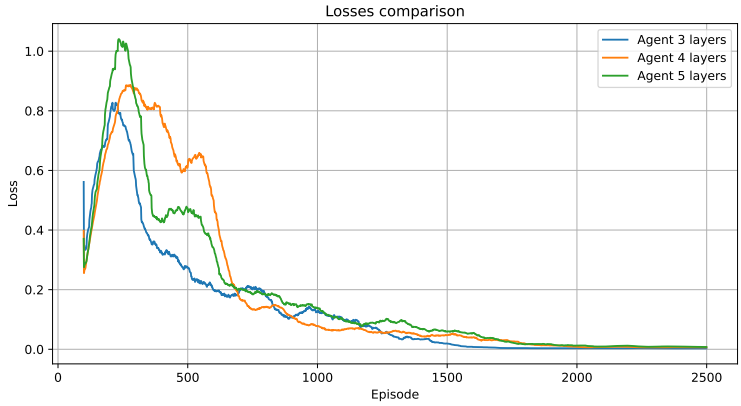

**Observation on the test** The 4 layers agent has more difficulties to find optimality. This can be seen also from the losses plot, where this agent reaches highest values, if compared with the others in the deterministic scenario. In the nondeterministic one, the 4 layers agent shows a delay on reaching optimal rewards values.

### 5.3.3 Buffer size equal to 20000

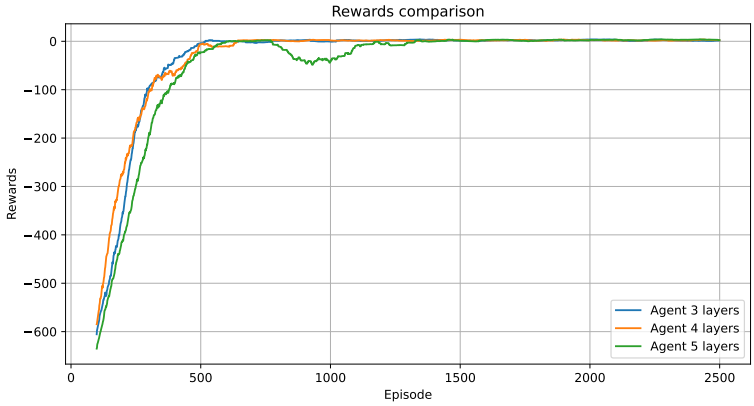
Deterministic

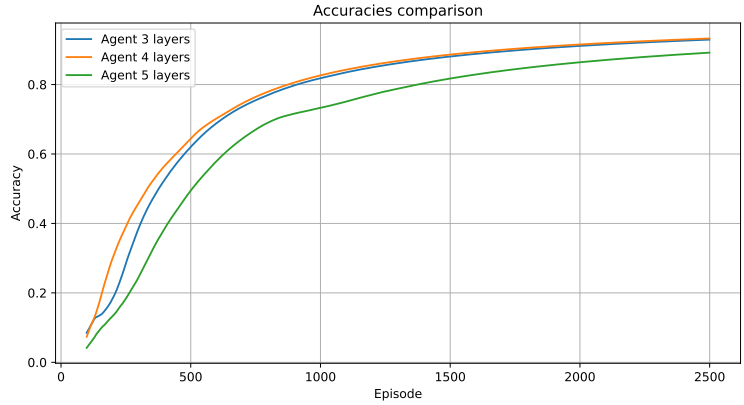


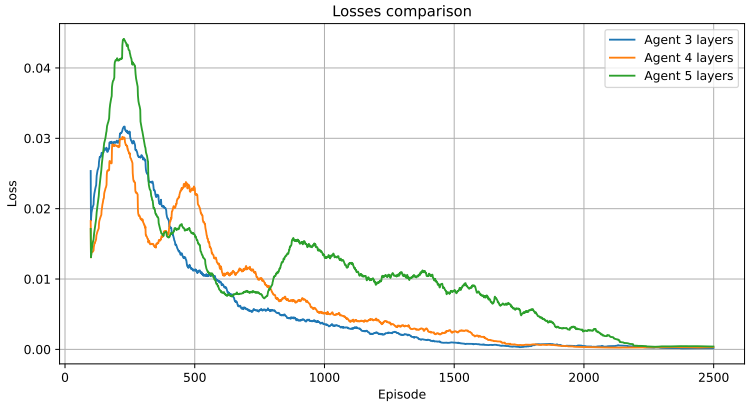



Nondeterministic:



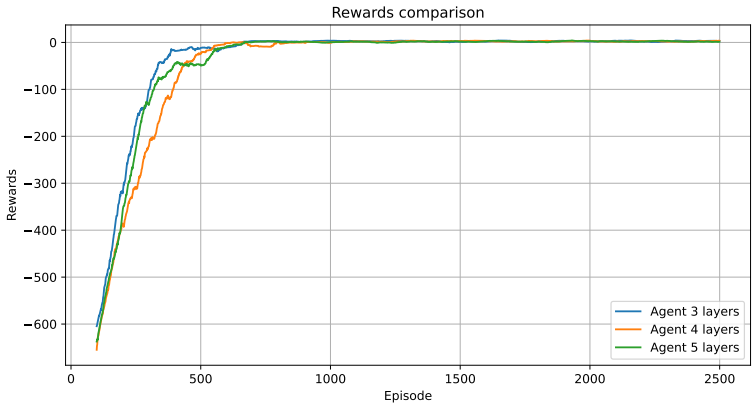


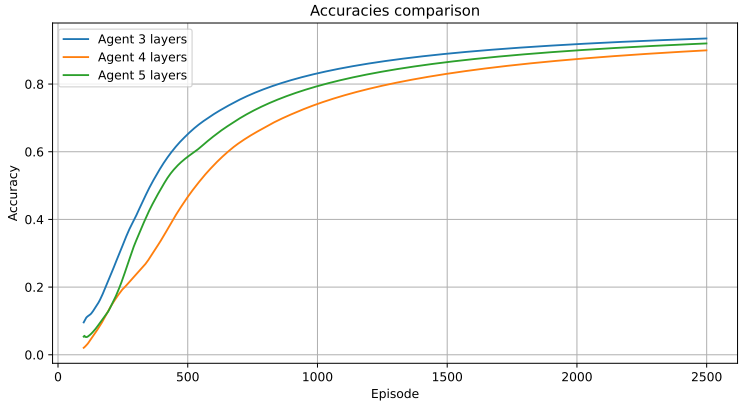



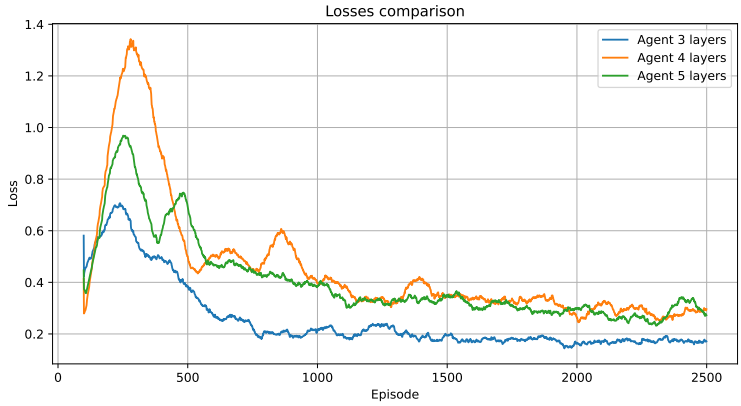

**Observation on the test** For this size of Replay Buffer, it can be seen that the behavior of all the agents is quite similar. The only remarkable difference is with the nondeterministic 5 layers agent, which has a little flexion around 1000 episodes.

### 5.3.4 Buffer size equal to 100000

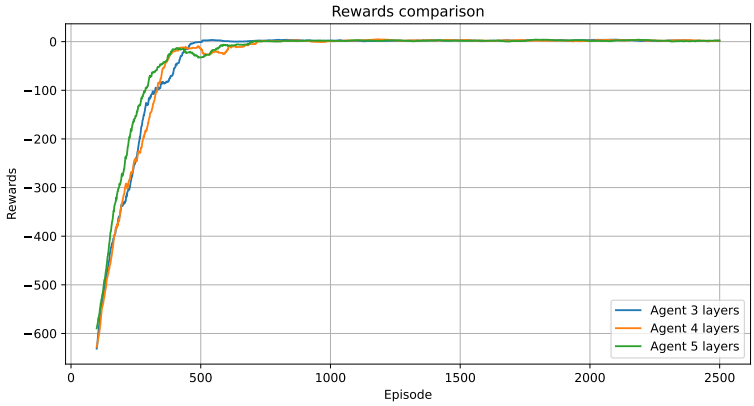
Deterministic

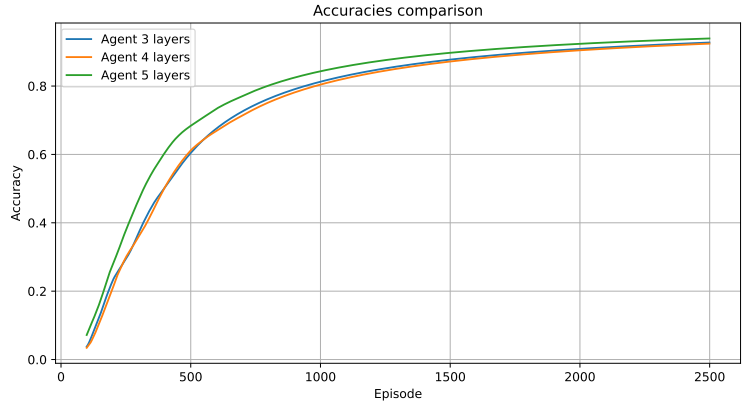


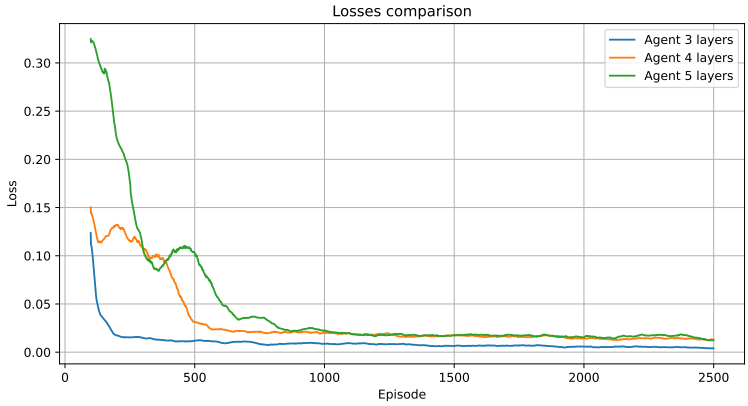



Nondeterministic:






**Observation on the test** With respect to the previous tests, a size of 100000 elements for the buffer brings the best results in all the metrics for all the agents. Differently from the "flat-like" trend of the nondeterministic case, the deterministic agents show oscillating course around the value of the loss that seems to be their convergence.

## 5.4 General observation

An important difference with this solution between the deterministic case and nondeterministic is the loss reached by all the agents. The nondeterministic formula leads to have a lower loss with respect to the deterministic one.

# 6 Conclusions

The main goal of the project was about showing that different implementation of the Q learning can, with theoretical and practical differences, train a model for agents that is capable to solve a certain environment.

With the choice of this particular environment, it emerges that the tabular solution is able to find in an enough precise and quick way an optimal policy in the situation of discrete states. Thanks to the nature of the environment, that provides a restricted set of possible states where the agent can be with a limited range of actions executable, there is no need to perform complex operations or storing many and heavy information inside the data structures used for the learning. This suit very well with the general idea of the "traditional" Q Learning, where each combination of state-action is stored inside a table; in the proposed implementation, it is used a matrix. So, by accessing the elements of such matrix, it can be read and updated the Q-value of a specific tuple state-action in a way like this:

```
self.table[state][action] = (1-alpha)*self.table[state][action]+ alpha*(reward+ gamma
* np.max(self.table[next_state])) .
```

The reported line of code, represents the update of the Q-value for specific state and action during the learning, where the variable `table` is the virtual representation of the previously cited matrix.

On the other hand, even if there might be some slightly slower trends, also the solutions with the neural network easily learn how to solve the environment. In general, these approaches expose a more gradual and slower curve of learning, with the need of more episodes for reaching the optimality in terms of all the metrics analyzed during the tests. More in depth, instead of memorizing the Q-values for each possible state-action combination, the usage of a supplementary memory (represented with the `Replay Buffer`) is dedicated to storing information like the states, the rewards, the actions and so on. With all these experiences saved inside the buffer, it is possible train the network in order to use it as an "approximator" for estimating the current and the future Q-values. Another great difference relies on the usage of the functions for transforming the information retrieved from the minibatches. They are converted as tensors to generate an input for the neural network and for the loss function. So, in terms of code, the lines in charge of generating the estimation of the Q-value, to be compared with the current one, is the following:

```
target = ((1 - self.alpha) * q_value + self.alpha *(rewards+(self.gamma * next_q_value
* (1 - dones)))).detach() .
```

The variables `q_value` and `next_q_value` are obtained by retrieving the tensors from the neural networks.